

Order acceptance and scheduling under eligibility, availability, and budget constraints in distributed heterogeneous flow shop production

Fuli Xiong[✉], Lin Jing[✉], Chengfei Xiang[✉], Yitong Han[✉]

School of Information and Control Engineering, Xi'an University of Architecture and Technology, Xi'an 710055, China

ARTICLE INFO

Keywords:

Order acceptance and scheduling
Logic-based Benders decomposition
Constraint programming
Mixed integer linear programming
Local search

ABSTRACT

The distributed manufacturing model enhances production flexibility and efficiency. This study investigates the order acceptance and scheduling problem in distributed heterogeneous factories while considering constraints including order-to-factory eligibility, processing time availability, and budget limitations, with the objective of maximizing total profit. The problem involves four interrelated decisions: order selection, allocation, sequencing, and scheduling. To address this, we formulate a Manne-based mixed-integer linear programming (MILP) model and a constraint programming (CP) model. However, these methods become computationally impractical for large instances. To improve efficiency, we develop three logic-based Benders decomposition (LBBD) algorithms that iteratively solve a master problem (MP) for order acceptance and allocation and subproblems (SPs) for scheduling within each factory. A local search (LS) algorithm is integrated to enhance solution quality and convergence speed. Experimental results validate the effectiveness of the proposed approach. For small-scale instances, our method achieves optimality where MILP and CP fail. In large-scale cases, it reduces the optimality gap by 54.2% and 100% across two scenarios while cutting solution time by 20.6%, 84.2%, and 98.3% in three scenarios. Applied to real-world scenarios, it increases total profit by 4.9%, confirming its practical value. These results demonstrate that the proposed LBBD framework significantly outperforms MILP and CP, solving larger instances efficiently with superior solution quality. Additionally, sensitivity analysis highlights key trade-offs in due date flexibility, budget constraints, and order distribution for optimizing scheduling performance.

1. Introduction

With the intensification of market competition and the diversification of customer demands, many manufacturing enterprises are transitioning from traditional single-factory production mode to distributed multi-factory production mode. The distributed manufacturing mode significantly enhances system flexibility but also introduces challenges to production scheduling. Production decision-makers must simultaneously determine the assignment of orders to factories and the scheduling plans for orders within each factory. As a result, distributed production scheduling has attracted significant attention from both industry and academia. Distributed permutation flow shop scheduling problem (DPFSP), proposed by Naderi and Ruiz (2010), is one of the most typical problems in distributed production scheduling. In this problem, it is generally required that the order sequence at each stage of in each factory remains consistent, and that the buffers between two adjacent stages are usually assumed to be sufficiently large. However, from a practical perspective, sufficiently large buffers often imply that

the order sequences at different stages can be inconsistent. In other words, the scheduling problem for each factory can be abstracted as a non-permutation flow shop scheduling problem (NPFSP). Thus, the traditional distributed permutation flow shop scheduling problem can be extended to a distributed non-permutation flow shop scheduling problem (DNPFSP). In DNPFSP, multiple factories, machines, and orders are considered, with each order assigned to a single factory for processing, and the processing sequence on each machine can different (Xiong, Chen, Xiong, & Jing, 2025). This flexibility allows for a more efficient allocation of resources, enabling the system to accept more orders within the given deadlines. However, the combination of the distributed manufacturing mode and the non-permutation flexibility further increases the difficulty of solving the problem.

Most existing research on the Distributed Flow Shop Scheduling Problem (DFSP) assumes that all factories or production lines are identical, often referred to as the Distributed Identical Flow Shop Scheduling Problem (DIPFSP) (Paz & M., 2024). However, in real-world

* Corresponding author.

E-mail addresses: xiongfuli@xauat.edu.cn (F. Xiong), jljxd@xauat.edu.cn (L. Jing), xiangchengfei@xauat.edu.cn (C. Xiang), hanyitong@xauat.edu.cn (Y. Han).

<https://doi.org/10.1016/j.eswa.2025.127586>

Received 8 February 2025; Received in revised form 8 March 2025; Accepted 3 April 2025

Available online 11 April 2025

0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

industrial scenarios, the Distributed Heterogeneous Flow Shop Scheduling Problem (DHFSPP) is of significant practical relevance, particularly for manufacturing systems that span multiple locations. Factories in such systems are often specialized in different tasks or have varying production capacities, which presents unique scheduling challenges. The processing capabilities of different factories may vary considerably, requiring flexible scheduling solutions that can accommodate these differences. Importantly, the **eligibility constraints** between factories and orders is a critical aspect of DNPFSPP, as not all factories may be capable of processing every type of order. Each factory is typically specialized in specific types of tasks or product categories, and the processing requirements of each order may vary depending on the factory's capabilities and technological constraints. Therefore, assigning orders to the appropriate factories is crucial for ensuring the feasibility of the schedule and optimizing production efficiency. The DNPFSPP reflects the reality of distributed manufacturing networks, where each factory's setup and operational constraints are distinct. This problem is particularly relevant in industries such as automotive, electronics, and consumer goods manufacturing, where production is often distributed across geographically dispersed factories. Thus, the eligibility constraints between orders and factories is not only a technical requirement but also a strategic consideration that directly impacts the efficiency and success of distributed manufacturing operations. Addressing this eligibility constraints in scheduling models is essential for aligning operational constraints with production objectives, ensuring that the entire manufacturing network operates smoothly and efficiently.

Another critical aspect of the DNPFSPP is the inclusion of processing time windows. In real-world manufacturing systems, many orders are not processed immediately but are instead subject to various time constraints, such as release times or deadlines of orders. The time window constraints of an order can be interpreted as the **availability constraints** for that order. A time window of an order can be influenced by factors like customer requirements, supply chain constraints, equipment availability, and maintenance schedules. Incorporating processing time windows into the scheduling problem allows for more realistic models that better align with real-world scenarios. For instance, in the case of perishable goods, electronics manufacturing, or industries where products must be delivered by a certain deadline, having strict processing time windows is essential for ensuring that production is completed on time. If these windows are not respected, late deliveries can lead to customer dissatisfaction, increased costs, or even penalties, which can severely impact a manufacturer's profitability and reputation. Furthermore, time windows are often tied to contractual obligations or market demand fluctuations, making them an integral component of modern production scheduling systems. Incorporating time windows also allows for a better understanding of queueing systems within manufacturing plants and the interdependencies between machines and orders. As highlighted by research in the field (Han, Pan, & Gao, 2023), respecting time windows can optimize both throughput and resource allocation while minimizing the risk of bottlenecks and resource underutilization, thus improving the overall efficiency of production.

In today's customer-oriented market, the demand for customized products is rapidly increasing. The rise of mass customization and mass personalization has placed higher demands on manufacturers in terms of both production efficiency and flexibility (Chen, Ye, Ma, & Xu, 2024). Consequently, order acceptance and scheduling (OAS) of customer orders have become crucial for ensuring effective operations. While most existing studies on OAS focus on single-machine and identical parallel machine environments (Bartal, Leonardi, Marchetti-Spaccamela, Sgall, & Stougie, 2000; Ou, Zhong, & Wang, 2015), there is a growing need to address OAS in the context of heterogeneous factories. In real-world scenarios, factories often face the challenge of determining which orders to accept, considering factors such as production capacity, deadlines, and customer priorities. This decision-making process is particularly complex when numerous orders need to

be processed and resources (e.g., machines, labor) are limited. Order acceptance is especially critical in situations where there are **budget constraints** or limited production capacity. For instance, when a factory has a finite number of orders it can handle within a given time period, it must prioritize which orders to accept. By solving the order acceptance problem, manufacturers can optimize their decisions to maximize profits while meeting customer demands. This problem is particularly significant in industries where orders must be accepted within specific time frames, such as make-to-order systems, where customer orders are tailored to individual specifications and must be processed by strict deadlines.

Logic-based Benders decomposition (LBBD) has significant advantages as a decomposition algorithm for solving complex combinatorial optimization problems. By separating the problem into a master problem (MP) and one or more subproblems (SPs) (Hooker & Ottosson, 2003), LBBD efficiently tackles large-scale problems by exploiting their underlying structure. Unlike traditional Benders decomposition, which is limited to linear SPs, LBBD extends to any optimization problem, allowing the use of more flexible and problem-specific cutting planes. This enables the algorithm to handle a wider variety of problem types effectively. Additionally, the integration of CP techniques has further enhanced the efficiency and scalability of LBBD, particularly for scheduling problems. The ability of CP to model and solve intricate scheduling constraints allows LBBD to generate high-quality cuts and reduces the solution space more effectively, leading to faster convergence and improved solution quality for large and complex optimization problems.

For the above reasons, this paper explores a novel OAS problem: Order acceptance and scheduling under eligibility, availability, and budget constraints in distributed heterogeneous non-permutation flow shop production (OAS_EAB_DHNPFSPP). The objective is to maximize net profit, defined as the revenue of accepted orders minus production costs. Such comprehensive consideration of real-world production constraints provides a more practical framework for optimizing scheduling processes. Currently, most scheduling problems are addressed using metaheuristic methods. These methods have robust solving capabilities, enabling them to obtain high-quality feasible solutions with relatively low computational costs (see Section 2.1). However, for large-size scheduling problems, the search space of metaheuristic methods increases dramatically, resulting in reduced search efficiency and potentially requiring more time to find satisfactory solutions. Thus, it is crucial to develop systematic approaches that can effectively obtain near-optimal solutions. It is worth noting that the OAS_EAB_DHNPFSPP problem can be naturally decomposed into three sub-problems: order acceptance, task assignment, and scheduling. In this context, the LBBD approach is well-suited for addressing this problem. Moreover, when applying LBBD to OAS_EAB_DHNPFSPP, the master problem and subproblems resulting from the decomposition exhibit relatively low computational complexity, albeit with the possibility of requiring a high number of iterations. Therefore, we propose combining local search techniques with LBBD to strengthen the Benders cuts and provide improved lower bounds. This paper explores a novel DHNPFSPP problem that accounts for budget constraints, processing time windows, and factory order-to-factory eligibility. To efficiently obtain optimal or near-optimal solutions with measurable quality, we first formulate a MILP model and a CP model. Then, we design three LBBD algorithms based on the specific structure of the problem. To the best of our knowledge, this is the first time that LBBD algorithms have been applied to the DHNPFSPP problem. The key contributions of this study are summarized as follows:

- This study investigates a novel order acceptance and scheduling problem in distributed heterogeneous flow shops. The problem involves determining which orders to accept, assigning orders, and sequencing them across multiple stages within specified time windows. It incorporates various practical considerations such

as budget constraints, processing costs, and eligibility criteria. To address this problem, both a Manne-based mixed integer programming (MILP) model and a constraint programming (CP) model are developed.

- Three LBB algorithm algorithms are proposed for solving the OAS_EAB_DHNPFSP problem. Specifically, several critical components are integrated into our LBB algorithm framework, including the use of CP-based initialization to provide high-quality upper and lower bounds, the application of various methods to solve the SSP, and the incorporation of LS techniques to enhance solution quality.
- The performances of the models and the proposed LBB algorithm algorithms for OAS_EAB_DHNPFSP are compared, and the effectiveness and efficiency of the models and the LBB algorithm algorithms are verified. In small-size instances, the proposed LBB algorithm decomposition methods achieve the best solution quality, demonstrating their superior performance. For large-size instances, the LS-enhanced LBB algorithm approach outperforms other methods, highlighting its effectiveness in handling more complex problem sizes.
- The LBB algorithm algorithm was applied to an industrial case study in automotive camera lens manufacturing. The results indicate that the LBB algorithm algorithm increased the profit from 2308 to 2421, the number of accepted orders from 18 to 19, and the budget utilization rate from 90.10% to 96.60%. These improvements demonstrate the effectiveness of the algorithm in enhancing economic efficiency in industrial applications.
- The sensitivity analysis underscores the importance for manufacturers to balance due date flexibility, budget, and order distribution in order to optimize scheduling efficiency. Tighter due dates increase complexity, requiring more resource allocation, while larger budgets introduce greater coordination challenges, potentially reducing solution quality. Overloading factories with too many orders can lead to inefficiencies. Managers must strategically manage these factors to ensure feasible, high-quality schedules while optimizing resource utilization, ultimately improving production processes in distributed manufacturing systems.

The remainder of this paper is organized as follows. In Section 2, we introduce the related work on DPFSP and LBB algorithm. In Section 3, the OAS_EAB_DHNPFSP problem is formally defined, and the corresponding MILP and CP models are established. Subsequently, in Section 4, we propose the LBB algorithm and hybrid LS and LBB algorithm frameworks for OAS_EAB_DHNPFSP. Next, computational experiments, practical engineering applications, and sensitivity analysis are presented in Section 5. Finally, conclusions are drawn in Section 6.

2. Literature review

In this section, first, we will present a literature review on three scheduling problems that are highly related to OAS_DHFSEAB: Distributed flow shop scheduling problem (DFSP) (Section 2.1), flow shop problem with available time windows (FSSP_PTW) (Section 2.2), and order acceptance and scheduling problem (OASP) (Section 2.3). Table 1 provides a summary of FSSP_PTW, DFSP, OASP and this study. Finally, we will review the recent development of LBB algorithm techniques for manufacturing scheduling problems in Section 2.4, as well as highlight the differences between our work and the existing literature.

2.1. Distribution flow shop scheduling problem (DFSP)

The Distributed Flowshop Scheduling Problem (DFSP) is an extension of the classical flowshop scheduling problem and has rapidly become a prominent research topic in the scheduling field. It is also an active area of research within Operational Research (OR) (Paz & M., 2024). DFSP involves assigning tasks to multiple factories or production lines, where each factory has independent resources and

flowshops. Jobs may flow between factories, requiring careful consideration of factory coordination, transportation times, and resource allocation. Moreover, variations in equipment and processing capabilities across factories further increase the complexity of the scheduling problem. Depending on whether all factories are identical, DFSP can be categorized into two types: distributed homogeneous permutation flowshops and distributed heterogeneous permutation flowshops (DHPF). In real-world manufacturing environments, machines often degrade or improve over time, making DHPF a more accurate representation of actual production settings.

Meng and Pan (2021) addressed the Distributed Heterogeneous Permutation Flowshop Scheduling Problem (DHPFSP), incorporating lot streaming and sequence-dependent setup times, aiming to minimize the maximum completion time across factories. They developed five heuristic algorithms and an enhanced artificial bee colony algorithm (NEABC), showing the effectiveness of these approaches. Jing et al. (2021) proposed an improved iterated greedy algorithm (IG) and an iterated local search algorithm (ILS) to solve the DPFSP with uncertain processing times (DPFSP_UPT), improving efficiency through local search and dynamic destruction methods. Zhang et al. (2022) introduced the Matrix Cube-based Estimation of Distribution Algorithm (MCEDA) for energy-efficient distributed flowshop scheduling, targeting minimized makespan and reduced carbon emissions. Their hybrid initialization and model updating approach proved effective. Hou et al. (2022) focused on integrated distributed production and distribution with time window constraints, proposing a MILP model and an enhanced brain storm optimization algorithm that showed superior performance for large-scale problems. Mou et al. (2022) addressed energy-efficient distributed flowshop inverse scheduling, minimizing scheduling adjustments and energy consumption through a hybrid collaborative algorithm. Pan et al. (2023) examined the distributed lot-streaming permutation flowshop scheduling problem, using five metaheuristic algorithms and improving performance with the NEH heuristic. Their results indicated that the artificial bee colony algorithm outperformed others in minimizing completion time. Gogos (2023) tackled the distributed permutation flowshop scheduling problem with CP, developing models and heuristic algorithms to compute lower bounds and solve medium-scale problems. Huang et al. (2023) addressed DHPFSP with flexible machine speeds, proposing the Bi-Roles Co-Evolution (BRCE) algorithm, which avoided local optima through heuristic rules and local search strategies. Feng et al. (2024) proposed an iterated greedy algorithm and tabu memory-based local search to solve the DHPFSP with heterogeneous factories and delivery objectives, validating its effectiveness. Bai et al. (2024) studied DHPFSP under the makespan criterion, incorporating release dates and factory speeds. They developed a Mixed-MILP model and a Branch-and-Bound (B&B) algorithm, and proposed a Discrete Artificial Bee Colony (DABC) algorithm to handle large-scale instances. Li, Li et al. (2024) introduced a mixed-integer linear programming model and an iterated greedy algorithm with job allocation probability acceleration (IGP) to minimize makespan for the DHPFSP, achieving the best results in 535 out of 720 test instances. Li, Wang et al. (2024) addressed the multi-objective distributed heterogeneous welding shop scheduling problem (DHWSP), introducing a multitasking framework and knowledge transfer strategies. They proposed a memetic algorithm, which was validated through experiments on 20 instances, showing significant performance improvement. Zhao et al. (2024) studied the energy-aware distributed heterogeneous flexible flow shop scheduling problem (EADHFFSP), aiming to minimize tardiness and energy consumption. They developed a MILP model and a co-evolution algorithm with reinforcement learning, demonstrating its superior performance in experiments.

Most recently, Luo and Yan (2025) proposed a hybrid EDA/PPO approach for the stochastic distributed hybrid flow-shop scheduling problem (SDHFSP) to minimize makespan and enhance robustness. Zhang, Zhu et al. (2025) studied the energy-efficient distributed assembly no-wait flow-shop scheduling problem (EE_DANWFSP) with

Table 1
Summary of the reviewed literature.

Type	Paper	Subtype	Objective	Methodology
DFSP	Meng and Pan (2021)	DHPFSP_LS_CSDST	Makespan	NEABC
	Jing, Pan, and Gao (2021)	DPFSP_UPT	Makespan	IIG, ILS
	Zhang et al. (2022)	EE_DAPFSP	Makespan,TCE	MCEDA
	Hou, Fu, Gao, Zhang, and Ali (2022)	IDPDP_TW_FS	TWET	BSO
	Mou, Duan, Gao, Liu, and Li (2022)	DPFSP_IS	TEC	HCA
	Pan, Gao, Li, and Wu (2023)	DPFSP	Makespan	MH
	Gogos (2023)	DPFSP	Makespan	CP
	Huang, Li, Gong, Wang, and Wei (2023)	DHPFSP_FMS	Makespan and TEC	BRCE
	Feng, Zhao, Jiang, Tao, and Mei (2024)	DHPFSP_t	TT	IG
	Bai et al. (2024)	DHPFSP	Makespan	B&B, DABC
	Li, Li and Gao (2024)	DHPFSP	Makespan	MILP, IGP, IG
	Li, Wang, Gong and Ming (2024)	DHWSP	Makespan and TEC	Generalized multitasking framework, memetic algorithm
	Zhao, Yin, Wang, and Yu (2024)	EADHFFSP	TTD and TEC	MILP, DRLCEA,DDQN
	Luo and Yan (2025)	DHPFSP	Makespan	EDA/PPO
	Zhang, Zhu et al. (2025)	EE_DANWFSP	Makespan and TEC	MEDHEA
	Zuo, Zhao, and Yu (2025)	DHFSPMB	Makespan	MILP, TLSS
	Zhang, Han, Wang, Li and Gao (2025)	DBFSP with sequence-dependent setup time	Makespan and energy consumption costs	A-DABC
	Xiong et al. (2025)	DHNPFSP_TWT	TWT	MILP, CP, TAE, AE_CP
FSSP_PTW	Yeung, Oğuz, and Cheng (2004)	FSP_ET	TWET	B&B
	Luo, Huang, Zhang, and Dai (2011)	two-stage HFSP	Makespan	GA
	Zhou, Che, and Yan (2012)	FSP with time window constraints	Makespan	MIP
	Han, Shi, and Yao (2014)	HFSP with time window	TWET	GA
	Bin and Bo (2020)	PFSP with learning effect and resource allocation	TWET	MILP
	Jing, Pan, Gao, and Wang (2020)	DPFSP_TWET	TWET	IGITE
	Ahmed and Younès (2021)	HFSP with basic blocking constraint	TWET	IG
	Hou et al. (2022)	IDPDP_TW_FS	TWET	EBSO
	Liu, Han, Wang, Liu, and Zhang (2024)	DHFm_lotcs_epsilon(TWET/DTW)	TWET	PL_IG_ITI
OASP	Rom and Slotnick (2008)	OASP in a single-machine system	Tardiness penalties	GA
	Og, Salman, Yalçın, et al. (2010)	OASP in a single-machine system with release dates, due dates, deadlines, and sequence-dependent setup times	Total revenue	GA
	Nobibon and Leus (2010)	OASP in a single-machine system	Total revenue	MILP, EA
	Wang, Xie, and Cheng (2013)	OASP in a two-machine flowshop system with due date and tardiness penalty	Total revenue	MILP, B&B, HA
	Lei and Guo (2015)	OASP in a flow shop system	Makespan, Total revenue	MILP, PNSA
	Ou et al. (2015)	OASP in a parallel machine system	Makespan	HA
	Noroozi, Mazdeh, Heydari, and Rasti-Barzoki (2018)	OASP in an integrated production-distribution system	Total revenue	MILP, EA
	Lin et al. (2019)	OASP in a flow shop system with release dates	Weighted number of tardy orders	ABC
	Riahi, Newton, Polash, and Sattar (2019)	COSP	TCT	deterministic constructive search algorithm

(continued on next page)

Table 1 (continued).

	Li and Ventura (2020)	OASP in a single-machine system with revenue, due date, and tardiness penalty	Total revenue	SBF, ASF, TIF, DPIA-LRSR
	Perea, Yepes-Borrero, and Menezes (2023)	OASP in a single-machine system	Total revenue	MILP, MHA
	Li, Li, Chen and Li (2024)	OASP in distributed assembly blocking flow shop system	TEC, TP, Makespan	KDNSGAI
	Fuli and Ziyang (2024)	OASP in flow shop system	Total net profit	MPIG
	Sampathi and Damodaran (2024)	OASP in job shop system	Total profit	SA
	Abbaas and Ventura (2024)	OASP in job shop system	Total profit	A multi-agent resource bidding algorithm
	Chen et al. (2024)	OASP in distributed heterogeneous manufacturing system with carbon caps, and each factory is configured with a single machine	Cost	MILP, B&B, LBBD
OAS_EAB_DHNPFSP	This study	OAS_EAB_DHNPFSP	Total revenue	MILP, CP, CP_LBBD _{MILP} , CP_LBBD _{CP} , CP_LBBD _{CP_LS}

makespan and energy consumption objectives, introducing the MED-HEA algorithm, which proved effective in experiments. Zuo et al. (2025) investigated the distributed hybrid flow-shop scheduling problem with machine breakdown (DHFSMB) and developed the TLSS algorithm, which outperformed existing methods. Zhang, Han et al. (2025) addressed the distributed blocking flow-shop scheduling problem with sequence-dependent setup times, proposing the A-DABC algorithm to optimize energy consumption and makespan. Finally, Xiong et al. (2025) tackled the DHNPFSP to minimize total weighted tardiness (TWT), proposing MILP and CP models for small instances and the TAE algorithm for large instances, demonstrating superior performance.

2.2. Flow shop scheduling problem with processing time window constraints (FSSP_PTW)

The flowshop scheduling problem with processing time windows is crucial in modern manufacturing, directly impacting production efficiency, resource utilization, and order delivery. Time window constraints align scheduling problems with real-world production, enhancing flexibility, reducing delays, and improving competitiveness.

Research has evolved significantly in this field. Yeung et al. (2004) analyzed non-preemptive scheduling with due windows, proving NP-completeness and applying branch-and-bound methods. Luo et al. (2011) optimized two-stage hybrid flowshop scheduling using genetic algorithms. Zhou et al. (2012) introduced MILP models for robotic flowshops with time window constraints. Han et al. (2014) improved flexible flowshop scheduling with evolutionary algorithms. Bin and Bo (2020) optimized no-wait permutation flowshops by minimizing earliness, tardiness, and resource costs. Jing et al. (2020) developed an iterated greedy algorithm (IGITE) for distributed flowshops, effectively managing idle times. Ahmed and Younès (2021) refined iterated greedy methods for hybrid flowshops with blocking constraints. Hou et al. (2022) addressed integrated production and distribution scheduling with time windows using a MILP model and optimization algorithms. Liu et al. (2024) proposed a penalty lot-assisted iterated greedy algorithm (PL_IG_ITI) for distributed hybrid flowshop scheduling, significantly improving timeliness and minimizing weighted earliness and tardiness. These studies highlight the significance of time window constraints in optimizing flowshop scheduling. The development of MILP models, genetic algorithms, and iterated greedy methods enhances scheduling efficiency, resource utilization, and on-time delivery in complex manufacturing environments.

2.3. Order acceptance and scheduling problem (OASP)

The problem of Order Acceptance and Scheduling (OAS) holds significant theoretical and practical value in the field of production

scheduling. Particularly in resource-constrained manufacturing environments, determining how to accept orders effectively to maximize profit and enhance production efficiency has become a critical issue. The OAS problem impacts not only production efficiency but also resource utilization and profitability, thus attracting extensive attention and in-depth exploration from researchers.

Early research on the Order Acceptance and Scheduling (OAS) problem laid a strong foundation for subsequent advancements. Rom and Slotnick (2008) explored large-scale OAS problems using genetic algorithms, demonstrating the effectiveness of these intelligent algorithms in handling the complexity of order selection and scheduling. Their study showed significant improvements in resource utilization and profitability for enterprises. Og et al. (2010) extended this work by applying genetic algorithms in Make-to-Order (MTO) systems, improving the efficiency and responsiveness of production processes in more complex environments. Nobibon and Leus (2010) proposed exact algorithms for single-machine OAS problems, illustrating that optimal solutions can be achieved in small-scale settings. As the research progressed, Slotnick (2011) introduced a dynamic programming model for OAS under resource constraints, providing a systematic framework for selecting and scheduling orders within tight time windows, a core issue in modern production environments. Wang et al. (2013) investigated the OAS problem in a two-machine flowshop, focusing on maximizing total net revenue by determining which orders to accept and how to schedule them. They formulated the problem as a MILP model and developed both a heuristic and a branch-and-bound algorithm. Results showed that the B&B algorithm effectively handles small instances, while the heuristic demonstrates strong performance on larger instances. Shabtay, Gaspar, and Kaspi (2015) provided a comprehensive review of various OAS models involving order rejection, offering a theoretical framework for addressing the OAS problem across different scheduling contexts. Later, Ou et al. (2015) developed an improved heuristic algorithm for parallel machine OAS, achieving high efficiency and accuracy in large-scale scheduling by incorporating neighborhood search and local optimization strategies. Lei and Guo (2015) examined the OAS problem in a flow shop environment, aiming to minimize makespan while maximizing net revenue. They proposed a MILP model and a parallel neighborhood search algorithm, with computational experiments demonstrating the effectiveness and competitiveness of the proposed method.

Recent studies have increasingly focused on dynamic and large-scale applications of the OAS problem. In practical applications, Noroozi et al. (2018) demonstrated that optimizing order acceptance and distribution scheduling in logistics can reduce transportation costs and improve customer satisfaction. Lin et al. (2019) applied the Artificial Bee Colony (ABC) algorithm to handle dynamic scheduling with uncertain order release times, improving scheduling performance in unpredictable environments. Riahi et al. (2019) presents a tailored

approach to the Customer Order Scheduling Problem (COSP) by incorporating problem-specific heuristics into both constructive and perturbative search algorithms, resulting in improved solutions compared to existing state-of-the-art algorithms. Li and Ventura (2020) examined the OAS problem in a single-machine make-to-order system, focusing on maximizing revenue minus tardiness penalties. The study proposed three mathematical models and exact algorithms, with experimental results indicating that the Lagrangian relaxation-based method offered the highest computational efficiency. Similarly, Perea et al. (2023) investigated the OAS problem, incorporating revenues and various costs. They proposed a MILP model and a metaheuristic algorithm, with results demonstrating the metaheuristic's efficiency in solving large-scale instances and providing valuable managerial insights. Li, Li, Chen et al. (2024) proposed a knowledge-driven non-dominated sorting genetic algorithm-II (KDNSGAI) to solve the distributed assembly blocking flowshop scheduling problem with order acceptance (DABFSP_OAS), optimizing makespan, total energy consumption (TEC), and total profit (TP). Experimental results showed its superiority over existing multi-objective algorithms. Fuli and Ziyang (2024) addressed the OAS problem in flow shops, introducing a Multi-Thread Parallel Iterated Greedy algorithm (MPIG) that improved search efficiency and outperformed other optimization methods, achieving at least an 11% performance gain over heuristic approaches. Focusing on high-mix, low-volume manufacturing, Sampathi and Damodaran (2024) studied job shop scheduling with discrete and batch processing machines in a Make-To-Order (MTO) environment. They proposed a Simulated Annealing (SA) algorithm, which performed more efficiently than commercial solvers for larger problem instances. Similarly, Abbaas and Ventura (2024) tackled the joint OAS and assembly job shop scheduling problem using an agent-based approach with a combinatorial auction mechanism. Their method optimized job selection and scheduling while resolving capacity conflicts, demonstrating efficiency in solving large-scale problems. Finally, Chen et al. (2024) studied the OAS problem in distributed manufacturing, minimizing rejection, production, transportation, and tardiness costs using a logic-based Benders decomposition (LBBD) method combined with dynamic programming. While their research considered a single-machine system per factory, this paper extends the problem to a distributed flowshop environment with multiple machines modeled as a non-permutation flowshop. Additionally, real-world constraints such as budget limitations, processing time windows, and eligibility constraints are incorporated. The objective is to optimize order-to-factory assignments, sequencing within each factory, and completion times to maximize total profit.

As highlighted in the literature, most studies on the OAS problem primarily focus on single-machine settings. Even when multiple machines are considered, additional factors such as budget constraints, qualification requirements, and other operational limitations are often not adequately addressed. In contrast, real-world manufacturing environments typically involve multiple factories and machines, where these complexities play a critical role. Therefore, there is a significant gap in the research, particularly in terms of modeling the OAS problem in the context of multi-factory, multi-machine systems, while incorporating practical constraints that are essential in actual production scenarios.

2.4. LBBD and its applications

Benders decomposition is a classic technique for solving large-scale mixed-integer optimization problems by decomposing the problem into a master problem and subproblems, iteratively converging to the optimal solution (Benders, 1962). However, traditional Benders decomposition faces significant challenges when dealing with mixed-integer programming problems that have complex logical relationships. Rahmaniani, Crainic, Gendreau, and Rei (2016) highlighted that the difficulty in solving subproblems, particularly when there is intricate interaction between the master and subproblems, results in

a slower convergence rate, which in turn leads to inefficiencies in solving discrete and combinatorial optimization problems. To overcome these shortcomings, Hooker and Ottosson (2003) proposed LBBD, an extension of the traditional Benders decomposition. LBBD combines the decomposition strategy of master and subproblems with logical reasoning, allowing the generation of more effective "Benders cuts" tailored to the characteristics of different problems, thereby improving the performance of the traditional method. Harjunkski and Grossmann (2002) applied LBBD to complex scheduling problems in production planning, demonstrating its flexibility in decomposing and solving multi-stage optimization problems. Cordeau, Gendron, and Laporte (2017) explored the use of Benders decomposition in large-scale vehicle routing problems, revealing that it is particularly effective for instances with complex constraints.

LBBD has gained significant attention due to its success in solving complex optimization problems across various fields. Papadakos (2008) pioneered the integration of Benders decomposition into large-scale airline crew scheduling problems, demonstrating considerable computational efficiency and practical applicability in this domain. Subsequently, Boland, Savelsbergh, and Waterer (2012) explored the application of LBBD to mixed-integer nonlinear programming (MINLP) problems. Their work showcased the potential of LBBD in solving non-convex optimization problems effectively, particularly in challenging industrial settings. Building on these foundational studies, Laporte, Vena, and Feliu (2015) provided a comprehensive review of recent advances in Benders decomposition, highlighting its applications in transportation, supply chain management, and energy systems. Their review emphasized the adaptability and relevance of Benders decomposition to a wide range of optimization challenges, solidifying its role in modern optimization techniques. Huang, Solomon, and Golden (2020) further extended the application of Benders decomposition by developing a heuristic method to optimize multi-objective maritime inventory routing problems. This approach successfully balanced routing costs with inventory levels, demonstrating the flexibility of Benders decomposition in addressing complex, multi-objective problems in logistics. More recent studies have focused on novel applications and environmental considerations. Li, Côté, Coelho, Zhang, and Zhang (2023) developed an exact LBBD method to address the order assignment and scheduling problem under processing and distribution time uncertainty in a multiple-factory manufacturing environment, where each factory is modeled as a single machine. Carla, Laurent, and Pierre (2023) introduced new modeling and decomposition strategies for applying LBBD to preemptive flexible job-shop scheduling, improving the efficiency of solving complex scheduling problems. Meanwhile, Chen et al. (2024) proposed an LBBD approach for order acceptance and scheduling in heterogeneous factories under carbon caps. Their exact algorithm effectively balanced production schedules with carbon emissions, providing a practical solution for managing manufacturing constraints in alignment with environmental regulations. Similarly to the work of Li et al. (2023), each factory in Chen et al. (2024) is modeled as a single machine.

The body of research reviewed demonstrates the increasing versatility of LBBD in solving complex optimization problems, from scheduling and routing to sustainability considerations, further solidifying its position as a critical tool in operational research. These studies collectively demonstrate that Benders decomposition and its variants, such as LBBD, offer powerful capabilities for solving discrete and combinatorial optimization problems, providing a flexible and efficient tool for addressing complex structured optimization challenges. Researchers continue to refine these algorithms and strategies, resulting in their widespread application in production planning, path planning, scheduling, and multi-stage optimization. However, to the best of our knowledge, no research has yet explored LBBD-based methods for DFSPs in which each factory is configured as a flow shop and each order assigned to a factory must pass through multiple stages.

This study examines the OAS problem in a distributed heterogeneous non-permutation flowshop environment while incorporating eligibility, availability, and budget constraints. The objective is to maximize net profit, defined as the revenue from accepted orders minus production costs. Existing research on the OAS problem predominantly focuses on single-machine or single-factory settings, often overlooking the complexities of real-world production, such as multi-factory coordination, diverse machine capabilities, and stringent operational constraints. Similarly, while DFSP and flow shop scheduling with time window constraints have been extensively studied, most approaches rely on heuristic algorithms. Although heuristics are effective in handling large-scale problems, they do not guarantee optimality or solution quality. To address these limitations, this study explores the applicability of the LBB — an exact optimization method — to the expanded OAS problem in a multi-factory, multi-machine setting. By integrating LBB into this context, this research aims to bridge the gap between heuristic-based approaches and exact optimization techniques, providing a more rigorous and theoretically grounded solution framework.

3. Problem description and formulation

In this section, the OAS_EAB_DHNPFSP is formally described (Section 3.1). Subsequently, a MILP model for OAS_EAB_DHNPFSP is formulated (Section 3.2). Finally, a CP model is developed (Section 3.3).

3.1. Problem description

The problem addressed in this study is defined as follows: A set J of J independent orders is to be scheduled across a set F of F heterogeneous factories. Each order $j \in J$ is characterized by a profit R_j and a processing time window $[r_j, d_j]$. Each factory $f \in F$ comprises a set M of M machines. The processing cost $V_{j,f}$ for an order j at a factory f depends on the processing time $p_{j,m}^f$ on machine m , with shorter processing times incurring higher costs. Orders assigned to a factory must pass through the machines in a specified technological order, starting with machine 1 and continuing sequentially to machine M . Eligibility constraints between an order and a factory is represented by $E_{j,f}$, which ensures alignment between the requirements of the order and the capabilities of the factory. Unlike the traditional permutation flow shop (PFS) scenario, this problem assumes a non-permutation flow shop environment, where the processing sequence of orders on machines can vary within each factory. This non-permutation characteristic reflects real-world production settings more accurately. The objective of this problem is to decide which orders to accept, assign the accepted orders to appropriate factories, sequence them on the machines within each factory, and determine the completion time for each order. The goal is to maximize the total profit, calculated as the total revenue from accepted orders minus the corresponding production costs.

To facilitate understanding, consider a simple example of OAS_EAB_DHNPFSP involving eight orders, three machines, and three factories (see Fig. 1). In this case, the optimal solution is as follows: Orders 5 and 8 are assigned to Factory 1; Order 4 is allocated to Factory 2; and Orders 3 and 7 are assigned to Factory 3. The processing sequence on each machine in Factory 1 is $\pi_1 = (8, 5)$; Factory 2 is assigned only one order, hence $\pi_2 = (4)$; and the processing sequence on each machine in Factory 3 is $\pi_3 = (3, 7)$. The production data for all orders are listed in Table 2. The profit of Factory 1 is calculated as $R_5 + R_8 - (q_{5,1} V_{5,1} + q_{8,1} V_{8,1}) = 233$. Similarly, the profit for Factory 2 is 112, and the profit for Factory 3 is 240. The total profit is the sum of profits from all factories, which equals 585, while the total cost is the sum of all order costs, amounting to 55, which does not exceed the budget of 57. The scheduling results for Factory 1 are shown in Fig. 2(a), while the scheduling results for Factory 2 and 3 are shown in Fig. 2(b) and (c), respectively.

In this study, several assumptions are done as follows.

- All machines are available at time zero.
- Each machine can process at most one order at a time.
- The buffers between two adjacent stages are infinite.
- Setup times are neglected.
- The processing times on all machines across all factories, the production costs of each order in different factories, and the eligibility constraints between orders and factories are predefined constants. Additionally, the budget is also considered a known constant.
- Order processing cannot be preempted before it is finished.
- No order visits a machine more than once.

3.2. MILP model for OAS_EAB_DHNPFSP

Highlighted by Naderi, Ruiz, and Roshanaei (2023), a Manne-based MILP model for OAS_EAB_DHNPFSP is formulated. For convenience, the notations used throughout the paper are listed as follows:

Indices and input parameters

J	Number of orders;
M	Number of machines;
F	Number of factories;
$\mathcal{J}$	Set of orders, $\mathcal{J} := \{1, \dots, J\}$;
$\mathcal{M}$	Set of machines, $\mathcal{M} := \{1, \dots, M\}$;
$\mathcal{F}$	Set of factories, $\mathcal{F} := \{1, \dots, F\}$;
j, k	Index of orders;
m	Index of machines;
f	Index of factories;
R_j	Profit of order j ;
r_j	Release time of order j ;
d_j	Deadline of order j ;
$V_{j,f}$	Processing cost of order j in factory f ;
B	Total budget cost;
$p_{j,m}^f$	Processing time of order j on machine m in factory f ;
$E_{j,f}$	Boolean variable taking value 1 if order j is compatible with factory f ; 0, otherwise.
Ω	A sufficiently large number.

Decision variables

Z_j	Binary variable taking value 1 if order j is accepted; 0, otherwise;
Y_j	Binary variable;
$q_{j,f}$	Binary variable taking value 1 if order j is assigned to factory f ; 0, otherwise;
$X_{m,j,k}$	Binary variable taking value 1 if machine m processes order j before order k ; 0, otherwise;
$C_{j,m}^f$	Completion time of order j on machine m in factory f .

And then, the MILP model for OAS_EAB_DHNPFSP is formulated as follows.

$$\text{Maximize } \Phi = \sum_{j \in J} R_j Z_j - \sum_{j \in J} \sum_{f \in F} V_{j,f} q_{j,f} \quad (1)$$

subject to

$$\sum_{f \in F} q_{j,f} = Z_j, \forall j \in J, \quad (2)$$

$$q_{j,f} \leq E_{j,f}, \forall j \in J, f \in F, \quad (3)$$

$$\sum_{j \in J} \sum_{f \in F} V_{j,f} q_{j,f} \leq B, \quad (4)$$

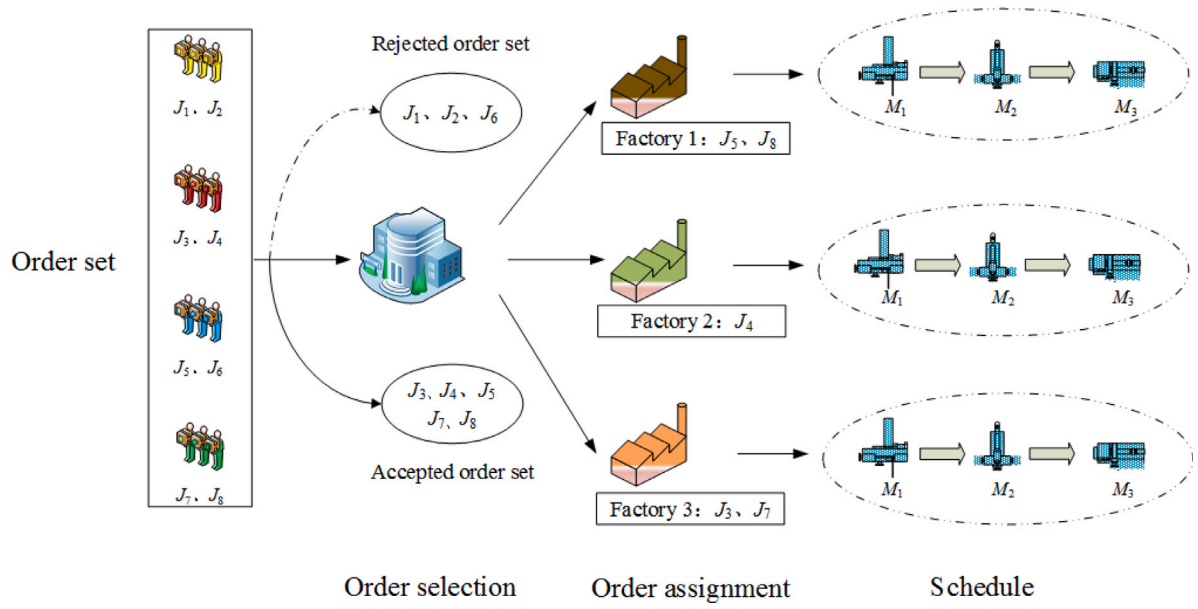


Fig. 1. Illustrate of a simple example for OAS_EAB_DHNPFS.

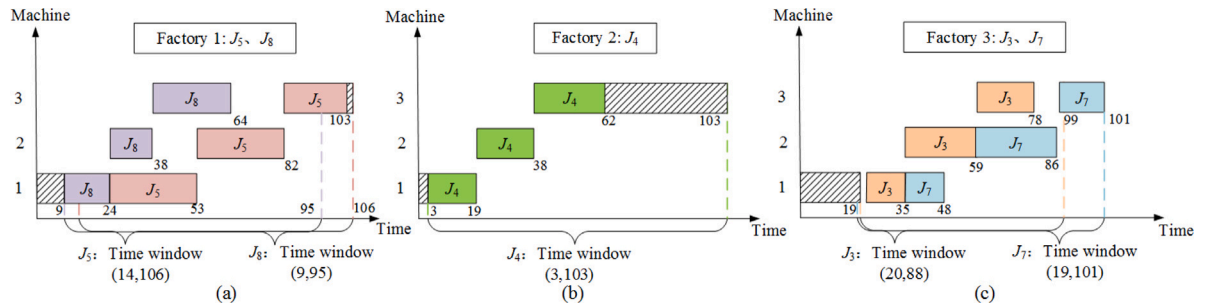


Fig. 2. Gantt chart of the example for OAS_EAB_DHNPFS. (a) Factory 1. (b) Factory 2. (c) Factory 3.

Table 2

The production data of a simple example for the OAS_EAB_DHNPFS.

Order	Factory	M_1	M_2	M_3	Cost	Profit	Time window	Eligibility	Budget
1	1	10	21	16	19	119	(13,94)	[0 0 1]	57
	2	22	26	10	10				
	3	26	17	28	5				
2	1	17	19	25	14	111	(12,92)	[1 0 0]	
	2	20	17	28	9				
	3	14	16	12	19				
3	1	19	13	15	20	132	(20,88)	[0 0 1]	
	2	15	16	21	12				
	3	13	24	19	5				
4	1	20	15	24	14	128	(3,103)	[0 1 0]	
	2	16	19	24	16				
	3	19	24	28	5				
5	1	29	29	21	5	123	(14,106)	[1 0 1]	
	2	19	26	16	17				
	3	19	24	27	12				
6	1	22	10	14	16	144	(6,93)	[1 0 0]	
	2	19	29	18	10				
	3	10	20	21	15				
7	1	27	15	30	8	128	(19,101)	[1 0 1]	
	2	12	22	26	12				
	3	13	27	15	15				
8	1	15	14	26	14	129	(9,95)	[1 0 0]	
	2	14	15	20	16				
	3	20	24	17	10				

$$C_{j,1}^f \geq (p_{j,1}^f + r_j)q_{j,f}, \forall j \in \mathcal{J}, f \in \mathcal{F}, \quad (5)$$

$$C_{j,m}^f \geq (C_{j,m-1}^f + p_{j,m}^f)q_{j,f}, \forall j \in \mathcal{J}, m \in \mathcal{M}, f \in \mathcal{F}, \quad (6)$$

$$C_{j,m}^f \geq C_{k,m}^f + p_{j,m}^f - \Omega(3 - X_{m,j,k} - q_{j,f} - q_{k,f}), \forall j, k \in \mathcal{J}, m \in \mathcal{M}, \\ \times f \in \mathcal{F}, j > k, \quad (7)$$

$$C_{k,m}^f \geq C_{j,m}^f + p_{k,m}^f - \Omega(2 + X_{m,j,k} - q_{j,f} - q_{k,f}), \forall j, k \in \mathcal{J}, m \in \mathcal{M}, \\ \times f \in \mathcal{F}, j > k, \quad (8)$$

$$C_{j,m}^f \leq \Omega q_{j,f}, \forall j \in \mathcal{J}, m \in \mathcal{M}, f \in \mathcal{F}, \quad (9)$$

$$C_{j,M}^f - d_j \leq \Omega(1 - Y_j), \forall j \in \mathcal{J}, f \in \mathcal{F}, \quad (10)$$

$$Z_j \leq Y_j, \forall j \in \mathcal{J}, \quad (11)$$

$$C_{j,m}^f \geq 0, \forall j \in \mathcal{J}, m \in \mathcal{M}, f \in \mathcal{F}, \quad (12)$$

$$X_{m,j,k} \in \{0, 1\}, \forall m \in \mathcal{M}, j, k \in \mathcal{J}, \quad (13)$$

$$q_{j,f} \in \{0, 1\}, \forall j \in \mathcal{J}, f \in \mathcal{F}, \quad (14)$$

$$Z_j \in \{0, 1\}, \forall j \in \mathcal{J}, \quad (15)$$

$$Y_j \in \{0, 1\}, \forall j \in \mathcal{J}. \quad (16)$$

The objective function (1) seeks to maximize the total profit, which is the difference between the gross profit from the orders and the associated processing costs. Constraint (2) ensures that each accepted order is assigned to exactly one factory. Constraint (3) enforces that the selected factory satisfies the compatibility requirements of the respective order. Constraint (4) guarantees that the total processing costs of the assigned orders remain within the predefined budget. Constraint (5) ensures that the completion time of an order on the first machine is at least as long as the sum of its release time and processing time. Constraint (6) establishes that the completion time of each operation is no earlier than the completion time of the previous operation plus the processing time of the next operation. Constraints (7) and (8) prevent two operations from orders j and k from being processed on the same machine simultaneously. Constraint (9) sets the completion time of rejected orders to zero. Constraints (10) and (11) ensure that the completion time of each order does not exceed its specified deadline. Constraints (12) through (16) define the domains for the decision variables.

3.3. CP model for OAS_EAB_DHNPFSF

A different CP model is used to describe the OAS_EAB_DHNPFSF. CP is a powerful technique for solving combinatorial optimization problems. A CP model represents the problem through variables, domains, and constraints: variables denote the elements of the problem to be solved, domains define the possible values for each variable, and constraints restrict the legal relationships among variables. Unlike other optimization approaches, CP focuses on reducing the solution space through efficient constraint propagation mechanisms, thereby quickly identifying feasible or optimal solutions that satisfy all constraints. CP models are widely applied in scheduling, resource allocation, and graph layout, demonstrating significant advantages in solving discrete optimization problems with complex constraints. In this paper, highlighted by Naderi et al. (2023), the CP model for OAS_EAB_DHNPFSF is formulated as follows.

$$\text{Maximize } \Phi = \sum_{j \in \mathcal{J}} R_j Z_j - \sum_{j \in \mathcal{J}} \sum_{f \in \mathcal{F}} V_{j,f} q_{j,f} \quad (17)$$

subject to

$$\sum_{f \in \mathcal{F}} q_{j,f} = Z_j, \forall j \in \mathcal{J}, \quad (18)$$

$$q_{j,f} \leq E_{j,f}, \forall j \in \mathcal{J}, f \in \mathcal{F}, \quad (19)$$

$$\sum_{f \in \mathcal{F}} \sum_{j \in \mathcal{J}} V_{j,f} q_{j,f} \leq B, \quad (20)$$

$$\text{PresenceOf} \left(Task_{j,m,f}^* \right) = q_{j,f}, \forall j \in \mathcal{J}, m \in \mathcal{M}, f \in \mathcal{F}, \quad (21)$$

$$Task_{j,m,f}^* = \text{IntervalVar} \left(p_{j,m}^f, \text{Optional} \right), \forall j \in \mathcal{J}, m \in \mathcal{M}, f \in \mathcal{F}, \quad (22)$$

$$\text{EndBeforeStart} \left(Task_{j,m,f}^*, Task_{j,m-1,f}^* \right), \forall j \in \mathcal{J}, m \in \mathcal{M} \setminus \{1\}, f \in \mathcal{F}, \quad (23)$$

$$\text{PresenceOf} \left(Task_{j,1,f}^* \right) = \text{PresenceOf} \left(Task_{j,m,f}^* \right), \forall j \in \mathcal{J}, \\ \times m \in \mathcal{M} \setminus \{1\}, f \in \mathcal{F}, \quad (24)$$

$$\text{NoOverlap} \left(Task_{j,m,f}^* : j \in \mathcal{J} \right), \forall m \in \mathcal{M}, f \in \mathcal{F}, \quad (25)$$

$$\text{If_then} \left(\text{PresenceOf} \left(Task_{j,0,f}^* \right), \text{StartOf} \left(Task_{j,0,f}^* \right) \geq r_j \right), \\ \forall j \in \mathcal{J}, f \in \mathcal{F}, \quad (26)$$

$$\text{If_then} \left(\text{EndOf} \left(Task_{j,M,f}^* \right) > d_j, Z_j = 0 \right), \forall j \in \mathcal{J}, f \in \mathcal{F}, \quad (27)$$

$$Z_j, q_{j,f} = \text{BinaryVar}(), \forall j \in \mathcal{J}, f \in \mathcal{F}. \quad (28)$$

Objective (17) aims to maximize the total profit, defined as the difference between the gross profit of the orders and the associated processing costs. Constraints (18) ensure that each accepted order is assigned to exactly one factory. Constraints (19) require that the assigned factory meets the compatibility requirements of the corresponding order. Constraints (20) guarantee that the processing costs for the assigned orders remain within the specified budget. Constraints (21) establish that a task is assigned to factory f if and only if $q_{j,f} = 1$. Constraints (22) define an interval variable for each order j assigned to machine m and factory f , representing the start and end times of the task, where the task is optional. Constraints (23) ensure that no order is processed by multiple machines simultaneously. Constraints (24) enforce that all operations of a given order are processed within the same factory. Constraints (25) prevent a machine from processing multiple orders simultaneously. Constraints (26) ensure that the start time of an order on the first machine is no earlier than its specified release time. Constraints (27) guarantee that the completion time of an order does not exceed its specified deadline. Constraints (28) define the domains of the decision variables.

4. The LBB algorithm for OAS_EAB_DHNPFSF

The following four decisions should be optimized for the OAS_EAB_DHNPFSF problem:

- Selection of orders from the order set $\mathcal{J} (Z_j)$,
- Assignment of orders to factories ($q_{j,f}$),
- Sequence of orders within each factory ($X_{m,j,k}$),
- Completion time of each operation ($C_{j,M}^f$).

As the problem size increases, a new LBB algorithm is designed to address the challenge where the CP and MILP models fail to provide measurable near-optimal solutions within the specified time. Given that the CP model can generate a good solution within a short time, the solution obtained by running the CP model for a time limit $t_{\max}^{CP}$ is used as the first step toward optimality before executing the LBB

algorithm. The term LB_{CP} is defined as the lower bound of the objective function value found by the CP model within t_{max}^{CP} , which serves as the lower bound for the entire problem. The value of t_{max}^{CP} is scenario-dependent. Similarly, the term UB_{CP} represents the upper bound of the OAS_EAB_DHNPFSF.

In this decomposition approach, the master problem considers both order acceptance and order-to-factory assignment, and we refer to this optimization problem as the Assignment Master Problem (AMP). The subproblems correspond to non-permutation flow shop feasibility scheduling problems and can be solved using either a MILP approach or a CP method, denoted as SSP_{MILP} and SSP_{CP} , respectively. If the AMP is solved optimally and all subproblems are feasible, the solution is considered optimal for the OAS_EAB_DHNPFSF. However, if at least one subproblem is found to be infeasible, feasibility cuts are generated and incorporated into the AMP. The updated AMP is then resolved in the subsequent iteration of the LBB algorithm. This iterative process continues until either an optimal solution is obtained or the predetermined time limit is reached. The pseudocode for the decomposition approach is provided in Algorithm 1.

In this study, we propose three LBB algorithms. The first two algorithms are denoted as CP_LBB_{MILP} and CP_LBB_{CP} . Both algorithms employ a CP model as a warm start before executing the LBB process. The key distinction between them lies in the method used to solve the subproblems: CP_LBB_{MILP} utilizes MILP, while CP_LBB_{CP} relies on CP for solving the subproblems. Furthermore, we introduce a LS to augment the CP_LBB_{CP} approach, resulting in a third variant, $CP_LBB_{CP_LS}$, which combines LBB with LS to improve the solution process.

Algorithm 1 CP_LBB for OAS_EAB_DHNPFSF.

Input: ϵ , problem data, maximum allowable computing time (*TimeLimit*), and the maximum run time for CP in the phase of warm start (t_{max}^{CP})

Output: Lower bound LB , upper bound UB , the duality gap Gap , computing time

- 1: Run CP for a pre-defined computation time t_{max}^{CP} to obtain LB_{CP} , UB_{CP}
- 2: Initialize lower bound LB , upper bound UB , and iteration number h : $LB \leftarrow LB_{CP}$, $UB \leftarrow UB_{CP}$, $h \leftarrow 0$, $Gap \leftarrow 100$
- 3: **while** (*TimeLimit* is not reached and $Gap > \epsilon$) **do**
- 4: $h \leftarrow h + 1$
- 5: Solve AMP and let its optimal assignment solution be q^h , Z^h , with the optimal value being Φ_h^{AMP} . Update: $UB \leftarrow \Phi_h^{AMP}$
- 6: **for** $f \in F$ **do**
- 7: Check feasibility of $SSP(q^h, Z^h)$
- 8: **end for**
- 9: **if** All $SSP(q^h, Z^h)$ are feasible **then**
- 10: The original problem is solved to optimality and $LB \leftarrow UB$
- 11: **break**
- 12: **else**
- 13: Generate *Benders no good cuts* (43) and add them into the AMP
- 14: Update the duality gap: $Gap \leftarrow 100 \times \frac{UB-LB}{UB}$
- 15: **end if**
- 16: **end while**
- 17: **return** LB , UB , Gap , and computing time.

In Algorithm 1, before initiating the LBB algorithm, the CP model is first executed for a duration of t_{max}^{CP} . If the original problem does not yield an optimal solution, the objective function's upper and lower bounds are derived from the best solution found by the CP model within the given time limit (first line). These bounds are then used as the initial bounds for the OAS_EAB_DHNPFSF. Following this, the initial values of the input parameters are provided (line 2). Next, the AMP is solved

using MILP with strong subproblem relaxations to obtain the order assignment solution q^h , Z^h , and the optimal objective value z_h^{AMP} (line 5), which serves as the upper bound for the OAS_EAB_DHNPFSF. The upper bound UB is updated to reflect the optimal objective value of the master problem. Subsequently, with q^h and Z^h fixed, the feasibility of all subproblems is checked. Each subproblem, denoted as SSP , is solved using either CP or MILP methods. If all subproblems are feasible, the lower bound LB is updated to the value of UB , i.e., z_h^{AMP} (lines 9–10). If any subproblem is infeasible, the lower bound remains unchanged, and the gap between UB and LB is recalculated. Benders' no-good cuts are generated and added to the AMP (lines 11–14). The AMP and subproblems are iteratively solved until the stopping criterion is satisfied. Finally, the best results obtained so far, including the optimal bounds and solution times, are returned.

This section provides a detailed explanation of each model. The AMP is presented in Section 4.1, and the subproblems of the decomposition approach are discussed in Section 4.2. Then, we introduce the *Benders no good cuts* applied in LBB for OAS_EAB_DHNPFSF in Section 4.3. Finally, to enhance the filtering of cuts, we propose a hybrid approach that combines LS with LBB, which is detailed in Section 4.4.

4.1. Assignment master problem (AMP)

The AMP is a relaxation of the original problem by removing the sequencing variables $X_{m,j,k}$. Therefore, the AMP is formulated as follows:

Objective function (1),

subject to Constraints (2)–(4), (14)–(15), *Benders no good cuts*(43), and

$$LB_{CP} \leq \Phi \leq UB_{CP} \quad (29)$$

Constraints (29) indicate that the objective value of AMP (denoted by Φ) lies within the lower and upper bounds obtained by running the CP model within a maximum allowed computation time t_{max}^{CP} .

4.2. Scheduling subproblems (SSPs)

After the AMP is solved, the values of the variables Z_j and q_{jf} are fixed, which allows for the determination of which orders are accepted and how they are assigned to factories. Consequently, each sub-problem (SP) is reduced to a single-factory scheduling problem, where the objective is to determine whether the processing of the orders can meet the required time windows. In other words, each sub-problem is essentially a feasibility problem. To address this, two models are developed for the subproblems using MILP and CP, respectively. Let $J_f^h := \{j : q_{j,f}^h = 1\}$ denote the set of jobs assigned to factory f at the h th iteration. The MILP model for subproblem is formulated as:

$$(SSP_{MILP}) \quad \text{Minimize} \quad 0 \quad (30)$$

subject to

$$C_{j,1} \geq p_{j,1}^f + r_j, \forall j \in J_f^h, \quad (31)$$

$$C_{j,m} \geq C_{j,m-1} + p_{j,m}^f, \forall j \in J_f^h, m \in \mathcal{M} \setminus \{1\}, \quad (32)$$

$$C_{j,m} \geq C_{k,m} + p_{j,m}^f - \Omega(1 - X_{m,j,k}), \forall j, k \in J_f^h, m \in \mathcal{M}, f \in F, j > k, \quad (33)$$

$$C_{k,m} \geq C_{j,m} + p_{k,m}^f - \Omega(X_{m,j,k}), \forall j, k \in J_f^h, m \in \mathcal{M}, f \in F, j > k, \quad (34)$$

$$C_{j,m} \geq 0, \forall j \in J_f^h, m \in \mathcal{M}, \quad (35)$$

$$C_{j,M} \leq d_j, \forall j \in J_f^h, \quad (36)$$

$$X_{m,j,k} \in \{0, 1\}, \forall j, k \in \mathcal{J}_f^h, m \in \mathcal{M}, f \in \mathcal{F}. \quad (37)$$

The objective function (30) is set to zero, indicating that the subproblem is a feasibility problem. Constraints (31) ensure that for each order j , the completion time on the first machine $C_{j,1}$ is at least the sum of the processing time on that machine $p_{j,1}^f$ and the release time r_j . Constraints (32) guarantee that the difference in completion times of order j on machines m and $m-1$ is at least the processing time $p_{j,m}^f$ on machine m . Constraints (33) and (34) prevent two operations, j and k , from being processed on the same machine simultaneously. Constraints (35) ensure that the completion time of each order is non-negative. Constraints (36) impose that the completion time of each order j does not exceed its deadline d_j . Constraints (37) define the domains of the decision variables.

As an alternative modeling method, a CP model for a subproblem is formulated as:

(SSP_{CP}) Objective function (30)

subject to

$$Task_{j,m}^* = \text{IntervalVar}(p_{j,m}^f), \forall j \in \mathcal{J}_f^h, m \in \mathcal{M}, \quad (38)$$

$$\text{EndBeforeStart} = (Task_{j,m}^* Task_{j,m-1}^*), \forall j \in \mathcal{J}_f^h, m \in \mathcal{M} \setminus \{1\}, \quad (39)$$

$$\text{StartOf}(Task_{j,0}^*) \geq r_j, \forall j \in \mathcal{J}_f^h, \quad (40)$$

$$\text{NoOverlap}(Task_{j,m}^* : j \in \mathcal{J}_f^h, \forall m \in \mathcal{M}, \quad (41)$$

$$\text{EndOf}(Task_{j,M}^*) \leq d_j, \forall j \in \mathcal{J}_f^h. \quad (42)$$

The objective function remains unchanged, representing a feasibility problem. Constraints (38) define interval variables, with one interval variable for each order on each machine. Constraints (39) ensure that an order cannot be processed on multiple machines simultaneously. Constraints (40) and (42) guarantee that the processing of each order adheres to its respective time window. Constraints (41) ensure that a machine cannot process more than one order at any given time.

4.3. Benders no good cuts

For the decomposition approach adopted in this study, if all subproblems are feasible for each machine, then the solution to the AMP is the optimal solution to the overall problem. Otherwise, Benders cuts are generated for the infeasible sub problems. This type of cut was proposed by Hooker, Ottosson, Thorsteinsson, and Kim (1999) and is referred to as “no good”. The *Benders no good cuts* are developed as follows:

$$\sum_{j \in \mathcal{J}_f^h} q_{j,f} \leq |\mathcal{J}_f^h| - 1, \forall f \in \mathcal{F}, \quad (43)$$

where $\mathcal{J}_f^h := \{j : q_{j,f}^h = 1\}$ denotes the set of orders assigned to factory f at the h th iteration.

Proposition 4.1. Inequalities (43) are valid *Benders no good cuts*.

Proof. To prove that Inequalities (43) represent valid *Benders no good cuts*, we need to show that these inequality exclude infeasible solutions while still maintaining the feasibility of the remaining solutions. Inequalities (43) ensure that, at the k th iteration ($k \geq h+1$), not all orders $j \in \mathcal{J}_f^h$ are assigned to factory f . In other words, at least one order $j \in \mathcal{J}_f^h$ must remain unassigned to factory f . Considering the following two cases:

Case 1: If all orders $j \in \mathcal{J}_f^h$ are assigned to factory f at the k th iteration ($k \geq h+1$), i.e., $q_{j,f} = 1$ for all $j \in \mathcal{J}_f^h$, then we have: $\sum_{j \in \mathcal{J}_f^h} q_{j,f} = |\mathcal{J}_f^h|$. This leads to a contradiction with the inequality: $\sum_{j \in \mathcal{J}_f^h} q_{j,f} \leq |\mathcal{J}_f^h| - 1$,

since $\sum_{j \in \mathcal{J}_f^h} q_{j,f} = |\mathcal{J}_f^h|$, which is clearly greater than $|\mathcal{J}_f^h| - 1$. Thus, such solutions are excluded.

Case 2: If at least one order $j \in \mathcal{J}_f^h$ is not assigned to factory f at the k th iteration ($k \geq h+1$), i.e., $q_{j,f} = 0$ for some $j \in \mathcal{J}_f^h$, then: $\sum_{j \in \mathcal{J}_f^h} q_{j,f} \leq |\mathcal{J}_f^h| - 1$. This satisfies the condition of Cuts (43), as it ensures that factory f must discard at least one order, fulfilling the inequality.

This completes the proof. $\square$

4.4. Hybrid decomposition method: CP_LBBD_{CP-LS}

For large-size problems, the LBBD method often suffers from weak cuts, which results in excessive iterations and prevents the method from finding a good solution in a reasonable amount of time. To address this issue, we propose a method that combines LS with LBBD, referred to as CP_LBBD_{CP-LS}. Specifically, when the subproblem of LBBD is infeasible, LS is invoked to transform the infeasible solution into a feasible one through reallocation and rejection of orders. During this process, cuts are generated for the infeasible combination and added to the master problem to reduce the feasible domain, thereby strengthening the cuts (Saab, 2021). For feasible allocations, a lower bound is generated and compared with the lower bound obtained from the CP warm start. At the end of the algorithm, the best lower bound is output to ensure both the efficiency of the algorithm and the quality of the solution. The details of the LS and the framework of CP_LBBD_{CP-LS} are presented in Sections 4.4.1 and 4.4.2, respectively.

4.4.1. LS

In this section, we propose a Local Search (LS) algorithm, the procedure of which is outlined in Algorithm 2. Specifically, the function `remove_order( $\tau_1, \mathcal{J}_s$ )` returns an order set obtained by removing an order τ_1 from the order set $\mathcal{J}_s$, while the function `add_order( $\tau_1, \mathcal{J}_i$ )` generates an order set by adding the order τ_1 to the order set $\mathcal{J}_i$.

To improve the efficiency of the local search process during the reallocation of orders, the following properties are identified and incorporated into the algorithm to accelerate its convergence:

Proposition 4.2. Consider an order j that is removed from an infeasible schedule associated with factory f . Let the cost of processing order j in factory f be denoted as $V_{j,f}$. If order j is transferred to a feasible factory f' , where the associated cost is $V_{j,f'}$, then, under the eligibility condition, the transfer is guaranteed to satisfy the budget constraint (4) if:

$$V_{j,f} \geq V_{j,f'}.$$

Proposition 4.3. Let j be an arbitrary order from factory f . If order j is to be transferred to another factory f' , then, under the eligibility condition, the transfer is guaranteed to satisfy the budget constraint (4) if:

$$V_{j,f} \geq V_{j,f'}.$$

Propositions 4.2 and 4.3 are relatively straightforward; therefore, their proofs are omitted for brevity.

Proposition 4.4. Consider an order k extracted from a factory f with a corresponding cost $V_{k,f}$. If this order is transferred to another factory f' , where the associated cost is $V_{k,f'}$, then, under the condition of eligibility constraints, the transfer is guaranteed to satisfy the budget constraint (4) if:

$$V_{k,f'} - 2V_{k,f} \leq B - \sum_{f \in \mathcal{F}} \sum_{j \in \mathcal{J} \setminus \{k\}} q_{j,f} V_{j,f}, \quad (44)$$

where B represents the budget limit, $\mathcal{F}$ is the set of factories, $\mathcal{J}$ is the set of orders, and $q_{j,f}$ is the decision variable indicating whether order j is assigned to factory f .

Proof. The total cost before transferring order k from factory f is $W + V_{kf}$, where W represents the total cost excluding order k , and V_{kf} is the cost of order k at factory f . After transferring order k to factory f' , the total cost becomes $W + V_{kf'}$. The cost difference due to the transfer is $V_{kf'} - V_{kf}$, representing the increase in cost.

To ensure that the total cost after the transfer does not exceed the budget B , the following condition must hold:

$$V_{kf'} - V_{kf} \leq B - V_{kf} - W.$$

This inequality states that the cost difference between transferring order k and its original cost at factory f must be less than or equal to the remaining budget, $B - V_{kf} - W$. Simplifying further, we derive the final condition:

$$V_{kf'} - 2V_{kf} \leq B - \sum_{f \in \mathcal{F}} \sum_{j \in \mathcal{J} \setminus \{k\}} q_{jf} V_{jf}.$$

Here, $\mathcal{F}$ represents the set of factories, $\mathcal{J}$ denotes the set of orders, and q_{jf} is a decision variable indicating whether order j is assigned to factory f . This condition ensures that the cost difference resulting from the transfer of order k from factory f to factory f' remains within the allowable budget, thereby maintaining the feasibility of the solution. $\square$

Incorporating these propositions into the local search heuristic ensures that only promising reallocation moves are evaluated, thereby minimizing redundant computations and enhancing the overall performance of the algorithm.

4.4.2. CP_LBBD_{CP-LS}

CP_LBBD_{CP-LS} builds upon the LBBD decomposition method by introducing a local search mechanism. In brief, when the subproblem SSP is infeasible, this method not only returns Benders feasibility cuts (43) to the master problem AMP, but also invokes a local search to reassign and reject orders. During this process, for infeasible allocation schemes, cuts are generated and added to the AMP to reduce the feasible region, thereby accelerating the algorithm's convergence. For feasible allocation schemes, a lower bound, denoted as LB_{LS} , is generated and compared with LB_{CP} . Throughout the computation, the best lower bound is output to ensure both the efficiency of the algorithm and the quality of the solution are improved. The pseudocode for CP_LBBD_{CP-LS} is provided in Algorithm 3. In Algorithm 3, the steps in the initial iterations correspond to lines 1–10 of Algorithm 1. Then, when the scheduling subproblem becomes infeasible, LS is invoked, and $LB_{LS}^{(h)}$ along with the cuts are output. The best lower bound LB so far is updated, and the cuts are returned to the AMP (lines 13–19). In the subsequent iterations, the steps are consistent with those in Algorithm 1.

5. Computational experiments

All experiments are conducted on a personal computer with a 2.4 GHz AMD A10-9600P RADEON R5 processor and 8 GB memory running on a Microsoft Windows 64-bit system. All models and algorithms are coded in Julia 1.6.7, linked with IBM ILOG CPLEX 12.10 including CPLEX and CP Optimizer engines. Specifically, the developed MILP formulations are solved by CPLEX engine called by JuMP package, whereas the developed CP formulations are solved by CP Optimizer engine called by PyCall package. All the parameters are set to their default values for both CPLEX and CP Optimizer engines.

5.1. Data generation

Since there is no standard dataset available for this research problem, we generate instances based on specific requirements to evaluate the performance of the proposed models. The dataset is generated based on various combinations of factories, orders, and machine quantities, resulting in a total of 36 small-size instances which include

Algorithm 2 LS for OAS_EAB_DHNPFPSP.

Input: Feasible factory order set $\mathcal{J}_i$, infeasible factory order set $\mathcal{J}_s$, feasible factory set $\mathcal{F}_1$, infeasible factory order set $\mathcal{F}_2$.

Output: Lower Bound LB_{LS} , cuts

```

1: for  $i \in \mathcal{F}_1$  do
2:   if  $\mathcal{F}_1 = \emptyset$  then
3:     for  $s \in \mathcal{F}_2$  do
4:       Calculate  $Q_j := R_j d_j$ 
5:       Sort the orders in  $\mathcal{J}_s$  by  $Q_j$  in non-decreasing order, yielding
         sequence  $\pi_s = (\tau_1, \tau_2, \dots, \tau_n)$ 
6:       Begin rejecting orders from sequence  $\tau$ , rejecting from 1 up to  $n$ 
         orders until  $\pi_s$  becomes feasible
7:       Record combinations of rejected infeasible orders and generate cuts
         based on inequalities (43)
8:       Calculate lower bound  $LB_{LS}$ 
9:     end for
10:   else
11:     for  $s \in \mathcal{F}_2$  do
12:       Calculate  $P_j := \sum_{m \in \mathcal{M}} p_{j,m}^f$ 
13:       Sort the orders in  $\mathcal{J}_s$  by  $P_j$  in non-increasing order, yielding
         sequence  $\sigma_s = (\tau_1, \tau_2, \dots, \tau_n)$ 
14:        $k \leftarrow 1$ 
15:       while  $k \leq \text{length}(\sigma)$  do
16:          $\tau_1 \leftarrow \sigma[k]$ 
17:         for  $i \in \mathcal{F}_1$  do
18:           if Order  $\tau_1$  is compatible with factory set  $\mathcal{J}_i$  and transferring
             order  $\tau_1$  to the factory set meets the budget constraints (44)
             then
19:              $\text{new\_}\mathcal{J}_s \leftarrow \text{remove\_order}(\tau_1, \mathcal{J}_s)$ 
20:              $\text{new\_}\mathcal{J}_i \leftarrow \text{add\_order}(\tau_1, \mathcal{J}_i)$ 
21:             if  $\text{new\_}\mathcal{J}_s$  and  $\text{new\_}\mathcal{J}_i$  are both feasible then
22:                $\mathcal{J}_s \leftarrow \text{new\_}\mathcal{J}_s$ 
23:                $\mathcal{J}_i \leftarrow \text{new\_}\mathcal{J}_i$ 
24:               break
25:             else
26:               Restore original order sets
27:               Record combinations of infeasible orders and generate
                 cuts based on inequalities (43)
28:             end if
29:           end if
30:         end for
31:        $k \leftarrow k + 1$ 
32:       Calculate lower bound  $LB_{LS}$ 
33:     end while
34:   if All  $\mathcal{J}_s$  removal is infeasible then
35:     Execute Lines 3–8 and perform the reject action
36:   end if
37: end for
38: end if
39: end for
40: return  $LB_{LS}$ , cuts

```

the following combinations: $\{2, 3, 4\} \times \{6, 10, 13, 15\} \times \{3, 4, 5\}$. These instances are used to evaluate the effectiveness of the MILP, CP, and LBBD models. In addition, there are 144 large-size instances generated to study the performance of the proposed decomposition algorithm for medium and large-size problems. These instances include the combinations: $\{2, 3, 4, 5\} \times \{16, 18, 20, 25, 30, 35, 40, 45, 50\} \times \{5, 6, 7, 8\}$. Each instance is named as F_J_M , where J , M , and F denote the number of orders, machines, and factories, respectively. For each instance, three different sets of experimental data were generated. As a result, there are a total of 108 small-size instances and 432 large-size instances. The computation time for small instances is limited to 600 s, while for medium- and large-size instances, it is limited to 1800 s.

In this study, to simulate the problem, we employ a detailed algorithm for generating and processing data. Specifically, the processing time of each order on different factories and machines, $p_{j,m}^f$, is randomly generated following a uniform distribution $U[10, 30]$. Additionally, the

Algorithm 3 CP_LBBD_{CP_LS} for OAS_EAB_DHNPFSP

Input: ϵ , problem data, maximum allowable computing time (*TimeLimit*), and the maximum run time for CP in the phase of warm start ($t_{\max}^{CP}$)

Output: Lower bound LB , upper bound UB , the duality gap Gap , computing time

- 1: Run CP for $t_{\max}^{CP}$ s to obtain LB_{CP} and UB_{CP}
- 2: Initialize lower bound LB , upper bound UB , and iteration number h : $LB \leftarrow LB_{CP}$, $UB \leftarrow UB_{CP}$, $h \leftarrow 0$, $Gap \leftarrow 100$
- 3: **while** (*TimeLimit* is not reached and $Gap > \epsilon$) **do**
- 4: $h \leftarrow h + 1$
- 5: Solve AMP and let its optimal assignment solution be q^h , Z^h , with the optimal value being Φ_h^{AMP} . Update: $UB \leftarrow \Phi_h^{AMP}$
- 6: **for** $f \in F$ **do**
- 7: Check feasibility of SSP(q^h , Z^h)
- 8: **end for**
- 9: **if** All SSP(q^h , Z^h) are feasible **then**
- 10: The original problem is solved to optimality and $LB \leftarrow UB$
- 11: **break**
- 12: **else**
- 13: Generate *Benders no good cuts* (43) and add them into the AMP
- 14: Execute Algorithm 2
- 15: Output LB_{LS}^h and cuts
- 16: **if** $LB_{LS}^h > LB$ **then**
- 17: $LB \leftarrow LB_{LS}^h$
- 18: **end if**
- 19: Generate strengthened *Benders cuts* and add them into the AMP
- 20: **end if**
- 21: Update the duality gap: $Gap \leftarrow 100 \times \frac{UB-LB}{UB}$
- 22: **end while**
- 23: **return** LB , UB , Gap , and computing time.

start time of each order, r_j , is also generated using a uniform distribution $U[1, 20]$. To simulate the due dates in real production environments, we introduce a time range that dynamically adjusts based on the number of machines and factories. The due dates for each order are determined as $d_j = T + \frac{\sum_{f \in F} \sum_{m \in M} p_{j,m}^f}{F}$, where T is a buffer term for the due date, generated from the interval $[\alpha \frac{30(n-1)}{F}, \alpha \frac{30n}{F}]$, and α is an adjustment factor. To ensure that each order can be processed in at least one factory, a compatibility matrix is generated. For the economic aspect, the potential profit of each order is randomly set within the range $[100, 150]$. The processing cost of each order is determined based on the processing time in each factory, considering the processing speed and related rankings. Finally, to calculate the budget of the project, the overall budget is estimated based on the minimum cost of processing each order across all factories. This value is multiplied by a coefficient β to determine a reasonable budget range that balances project profitability and production requirements. The computational parameters used in this study are summarized in Table 3.

5.2. Comparison of the models and the proposed LBBDs for small-size instances

In this section, we compare the performance of five methods (MILP, CP, CP_LBBD_{MILP}, CP_LBBD_{CP}, and CP_LBBD_{CP_LS}) on solving 81 small-size instances. These 81 instances were randomly generated, as described in Section 5.1. The abbreviations for the methods are provided in Table 4. The computational results are presented in Table 5, where “Gap” denotes the average gap, and “Time” represents the solution time. The calculation of Gap is given in Eq. (45). For convenience,

Table 3

Parameter settings for instances of different sizes.

Small-sizes: $J \in \{6, 8, 13, 15\}$, $M \in \{3, 4, 5\}$, $F \in \{2, 3, 4\}$
Large-sizes: $J \in \{16, 18, 20, 25, 30, 35, 40, 45, 50\}$, $M \in \{5, 6, 7, 8\}$, $F \in \{2, 3, 4, 5\}$
$p_{j,m}^f$: $U[10, 30]$
r_j : $U[1, 20]$
d_j : $T + \frac{\sum_{f \in F} \sum_{m \in M} p_{j,m}^f}{F}$
T : $U[\frac{30\alpha(n-1)}{F}, \frac{30\alpha n}{F}]$, $\alpha \in \{0.7, 0.5, 0.3\}$
R_j : $U[100, 150]$
$V_{j,f}$: $U[5 * \text{rank}, 5 * \text{rank} + 5]$
$E_{j,f}$: $\text{rand}\{0, 1\}$
B : $\beta \sum_{j \in J} \min_{f \in F} (V_{j,f})$, $\beta \in \{0.7, 0.5, 0.3\}$

Note: rank represents the descending order of the sum of processing times for an order across different factories.

LBBD1 refers to the method CP_LBBD_{MILP}, LBBD2 refers to the method CP_LBBD_{CP}, and LBBD3 refers to the method CP_LBBD_{CP_LS}.

In the evaluation of the five methods, we observed that when $J \leq 10$, all methods successfully obtained the optimal solution within 600 s, with solving times being comparable. This confirms the effectiveness of both the model and the LBBD method. However, when $J \geq 13$, the results varied. Specifically, the CP method failed to find the optimal solution within 600 s, and the MILP method also failed to reach the optimal solution in some cases (Gap = 10.89%). In contrast, the other three decomposition-based LBBD methods successfully obtained the optimal solution. Overall, all three LBBD methods found the optimal solution in 108 instances. This can be attributed not only to the fact that the CP method provides good upper and lower bounds within $t_{\max}^{CP}$, significantly reducing the search space, but also to the efficiency and applicability of the LBBD decomposition algorithm for solving such problems.

$$\text{Gap}(\%) = \frac{UB - LB}{UB} \times 100 \quad (45)$$

5.3. Comparison of the models and the proposed LBBDs for medium- or large-size instances

In this section, we compare the performance of the five methods across 432 medium- or large-size instances. Based on practical considerations, we defined three different scenarios by varying the due date coefficient (α) and the budget coefficient (β). Specifically, we considered the following three configurations: Scenario 1 with tight due dates and a high budget ($\alpha = 0.3$, $\beta = 0.7$), Scenario 2 with moderate due dates and a balanced budget ($\alpha = 0.5$, $\beta = 0.5$), and Scenario 3 with relaxed due dates and a low budget ($\alpha = 0.7$, $\beta = 0.3$). Due to the large volume of data, the detailed results are provided in Tables 18–20 of Appendix.

In Scenario 1, the computational results of the five methods across 432 medium- and large-size instances are organized based on different order sizes, machine counts, and factory numbers, and are presented in terms of Gap and Time. The results are provided in Table 6–8. Additionally, we compare the number of optimal solutions obtained by the five methods across different problem sizes, as shown in Fig. 3. From Table 6, it can be observed that as the order size increases from 16 to 50, the average solution time and average gap values of the different algorithms change significantly. Specifically, LBBD3 performs the best among the five methods, with the smallest Gap and the shortest solution time. In contrast, CP performs the worst when the order size is small ($J < 35$), but as the order size increases ($J > 35$), MILP’s performance gradually deteriorates, becoming the method with the largest Gap. For LBBD1 and LBBD2, the results indicate that when the order size ($J > 30$) increases, LBBD2 slightly outperforms LBBD1 in terms of Gap. This is primarily because, as the number of orders increases, the subproblem solved by the CP model is easier than that of MILP. Further analysis of the data in Tables 7 and 8 shows that as the number of machines and factories increases, LBBD3 consistently outperforms the other methods, achieving the smallest Gap values and the shortest

Table 4
The acronyms of five methods.

Acronyms	Method
MILP	The MILP model presented in Section 3.2.
CP	The CP model presented in Section 3.3.
CP_LBBD _{MILP} (LBBD1)	The LBBD method that uses the CP presented in Section 3.3 as a warm start and solves the subproblems of the LBBD using SSP _{MILP} outlined in Section 4.2
CP_LBBD _{CP} (LBBD2)	The LBBD method that uses the CP presented in Section 3.3 as a warm start and solves the subproblems of the LBBD using SSP _{CP} outlined in Section 4.2.
CP_LBBD _{CP_LS} (LBBD3)	The hybrid LBBD method presented in Section 4.4.

Table 5
Performance comparison of five methods for small-size instances.

<i>F</i>	<i>J</i>	<i>M</i>	Gap (%)					Time (s)				
			CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
2	6	3	0.00	0.00	0.00	0.00	0.00	0.07	0.25	0.07	0.06	0.063
		4	0.00	0.00	0.00	0.00	0.00	0.07	0.14	0.07	0.07	0.067
		5	0.00	0.00	0.00	0.00	0.00	0.07	0.15	0.08	0.07	0.074
	10	3	0.00	0.00	0.00	0.00	0.00	0.20	1.01	0.19	0.22	0.187
		4	0.00	0.00	0.00	0.00	0.00	0.22	4.95	0.20	0.21	0.202
		5	0.00	0.00	0.00	0.00	0.00	0.24	1.69	0.25	0.29	0.231
	13	3	15.36	0.00	0.00	0.00	0.00	600.19	18.25	105.59	108.93	100.81
		4	20.77	0.00	0.00	0.00	0.00	600.04	27.21	114.26	128.97	109.21
		5	14.03	0.00	0.00	0.00	0.00	600.11	36.08	101.77	104.59	104.48
	15	3	11.39	0.00	0.00	0.00	0.00	600.21	126.06	105.67	112.04	107.00
		4	22.13	10.89	0.00	0.00	0.00	600.05	600.05	151.26	192.00	123.29
		5	15.56	0.00	0.00	0.00	0.00	600.18	32.89	117.41	132.16	109.72
3	6	3	0.00	0.00	0.00	0.00	0.00	0.07	0.15	0.07	0.07	0.07
		4	0.00	0.00	0.00	0.00	0.00	0.08	0.13	0.07	0.07	0.08
		5	0.00	0.00	0.00	0.00	0.00	0.08	0.13	0.08	0.08	0.078
	10	3	0.00	0.00	0.00	0.00	0.00	0.30	0.36	0.19	0.19	0.253
		4	0.00	0.00	0.00	0.00	0.00	0.33	0.71	0.25	0.23	0.221
		5	0.00	0.00	0.00	0.00	0.00	0.72	0.57	0.42	0.45	0.463
	13	3	22.73	0.00	0.00	0.00	0.00	600.04	0.38	101.39	105.39	103.49
		4	34.62	0.00	0.00	0.00	0.00	600.19	1.06	107.93	115.89	112.41
		5	17.90	0.00	0.00	0.00	0.00	600.18	1.19	105.27	111.29	108.13
	15	3	24.25	0.00	0.00	0.00	0.00	600.04	24.27	120.22	137.88	138.06
		4	19.18	0.00	0.00	0.00	0.00	600.05	1.77	103.01	107.26	107.15
		5	14.90	0.00	0.00	0.00	0.00	600.13	42.05	103.88	108.05	109.68
4	6	3	0.00	0.00	0.00	0.00	0.00	0.10	0.08	0.08	0.08	0.082
		4	0.00	0.00	0.00	0.00	0.00	0.09	0.16	0.08	0.08	0.086
		5	0.00	0.00	0.00	0.00	0.00	0.12	0.15	0.10	0.10	0.096
	10	3	0.00	0.00	0.00	0.00	0.00	0.43	0.32	0.24	0.29	0.262
		4	0.00	0.00	0.00	0.00	0.00	0.42	0.52	0.28	0.32	0.279
		5	0.00	0.00	0.00	0.00	0.00	0.46	0.91	0.30	0.43	0.296
	13	3	23.50	0.00	0.00	0.00	0.00	600.05	0.36	101.51	104.74	105.08
		4	21.88	0.00	0.00	0.00	0.00	600.17	0.35	101.16	104.67	104.88
		5	0.00	0.00	0.00	0.00	0.00	2.66	0.43	2.58	2.83	2.26
	15	3	30.08	0.00	0.00	0.00	0.00	600.02	0.57	102.02	105.21	107.27
		4	25.49	0.00	0.00	0.00	0.00	600.02	1.08	118.02	133.78	109.95
		5	37.85	0.00	0.00	0.00	0.00	600.13	2.82	147.31	196.14	112.62
Average			10.32	0.30	0.00	0.00	0.00	283.57	25.81	53.15	58.75	52.18

solution times. In contrast, CP performs poorly, with its Gap values increasing significantly as the number of machines and factories grows. Although LBBD2 slightly outperforms LBBD1 in terms of Gap values in some cases, its solution time is longer, suggesting that LBBD2 incurs a higher computational cost when solving more complex problems. For the five methods' ability to obtain optimal solutions across 432 test cases, Fig. 3 provides a detailed comparison. From Fig. 3(a, b, c), it is evident that in all scenarios, LBBD3 consistently finds more optimal solutions than the other methods, while CP obtains the fewest optimal solutions. Specifically, when observing Fig. 3(a), it is noticeable that as the number of factories increases from 2 to 5, the number of optimal solutions found by LBBD3 significantly increases, from 32 to 65 (out of 108 test cases). In Fig. 3(b), for different numbers of machines, LBBD3 finds the optimal solution in nearly all cases, consistently achieving 50 optimal solutions (out of 108 test cases). On the other hand, Fig. 3(c) shows that when the order size ($J > 40$) exceeds a certain threshold, none of the five methods can find the optimal solution, demonstrating

a sharp increase in the difficulty of the problem. In summary, the experimental results from Scenario 1 clearly demonstrate that the LBBD outperforms both the CP and MILP methods, especially for large-size instances. Among the three LBBD methods, LBBD3 performs the best, surpassing both LBBD1 and LBBD2. This advantage can be attributed to the domain search strategy introduced in LBBD3, which significantly enhances the LBBD cuts and LB_{LS}. As shown in Fig. 3(d), LS provides better lower bounds than CP in each of the three runs for 144 instances, leading to improved lower bounds and overall solution quality.

The computational results for Scenario 2 (Tables 9–11, Fig. 4) show that LBBD3 consistently finds the optimal solution across all 432 test instances. In both Gap and computational time, LBBD3 outperforms all other methods. In contrast, CP performs the worst in all cases. While LBBD2 has a Gap very close to LBBD3, there is a small deviation at $J = 45$ ($Gap_{LBBD2} = 0.02$). However, LBBD3 has a clear advantage in computational time, consistently achieving lower values. For $J > 30$, the Gap for LBBD1 begins to increase. This trend matches the

Table 6

Computational results of five methods across different order sizes (Scenario 1).

J	Gap (%)					Time (s)				
	CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
16	29.58	1.96	0.31	0.49	0.00	1800.15	375.96	296.01	401.71	58.05
18	27.86	3.11	0.86	1.26	0.04	1800.15	473.29	419.28	526.09	113.16
20	28.75	6.62	4.04	4.25	0.47	1800.15	866.28	913.00	1034.37	265.19
25	27.09	12.77	10.02	10.21	3.02	1800.14	1331.27	1418.37	1446.43	781.28
30	26.68	19.47	14.75	14.70	6.15	1800.16	1800.23	1700.27	1714.10	1268.33
35	26.97	25.02	20.16	19.68	10.51	1800.18	1800.24	1800.00	1800.00	1684.42
40	27.26	28.29	22.94	22.32	14.51	1800.18	1800.31	1800.00	1800.00	1800.00
45	27.78	30.68	24.90	24.26	17.59	1800.18	1800.36	1800.00	1800.00	1800.00
50	28.72	33.34	28.06	27.34	21.61	1800.17	1800.43	1800.00	1800.00	1800.00
Average	27.85	17.92	14.00	13.83	8.21	1800.16	1338.71	1327.44	1369.19	1063.38

Table 7

Computational results of five methods across different machine sizes (Scenario 1).

M	Gap (%)					Time (s)				
	CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
5	26.40	16.56	11.75	11.50	6.86	1800.15	1314.66	1252.53	1298.60	1048.32
6	26.86	17.47	13.70	13.49	8.13	1800.15	1355.73	1333.98	1388.69	1087.79
7	27.97	18.14	14.19	14.18	8.15	1800.15	1343.82	1356.41	1391.40	1054.56
8	30.19	19.49	16.37	16.17	9.70	1800.19	1343.82	1366.82	1398.07	1062.85
Average	27.85	17.92	14.00	13.83	8.21	1800.16	1338.71	1327.44	1369.19	1063.38

Table 8

Computational results of five methods across different factory sizes (Scenario 1).

F	Gap (%)					Time (s)				
	CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
2	23.96	25.22	18.38	17.45	13.28	1800.13	1746.81	1539.72	1562.68	1333.18
3	26.64	18.63	14.66	14.64	8.57	1800.16	1348.96	1396.65	1459.39	1145.40
4	29.37	14.89	12.88	13.02	6.39	1800.17	1188.39	1253.26	1279.59	946.34
5	31.46	12.94	10.09	10.23	4.60	1800.19	1070.67	1120.12	1175.11	828.61
Average	27.85	17.92	14.00	13.83	8.21	1800.16	1338.71	1327.44	1369.19	1063.38

Table 9

Computational results of five methods across different order sizes (Scenario 2).

J	Gap (%)					Time (s)				
	CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
16	10.12	0.00	0.00	0.00	0.00	1762.77	17.81	2.76	4.64	3.22
18	8.71	0.00	0.00	0.00	0.00	1800.27	11.66	3.25	5.96	3.04
20	7.24	0.00	0.00	0.00	0.00	1725.45	18.83	6.29	7.28	3.04
25	4.76	0.10	0.00	0.00	0.00	1800.24	157.30	28.62	12.02	3.28
30	3.63	0.86	0.17	0.00	0.00	1800.28	640.81	137.40	16.18	5.72
35	2.77	0.57	0.18	0.00	0.00	1725.30	624.58	256.03	6.53	3.71
40	2.49	1.31	0.29	0.00	0.00	1762.75	992.52	321.97	32.45	11.00
45	2.72	1.12	0.83	0.02	0.00	1762.80	1131.67	632.09	85.17	27.84
50	2.56	1.11	1.01	0.00	0.00	1762.87	1054.52	627.43	23.48	15.89
Average	5.00	0.56	0.28	0.002	0.00	1766.97	516.63	223.98	21.52	8.53

Table 10

Computational results of five methods across different machine sizes (Scenario 2).

M	Gap (%)					Time (s)				
	CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
5	4.77	0.42	0.11	0.00	0.00	1783.60	442.49	186.07	9.76	3.71
6	4.75	0.53	0.12	0.00	0.00	1750.31	477.08	215.51	12.21	5.22
7	4.82	0.53	0.07	0.00	0.00	1766.93	501.01	184.54	13.68	4.02
8	5.64	0.77	0.80	0.01	0.00	1767.04	501.01	309.82	50.43	21.16
Average	5.00	0.56	0.28	0.002	0.00	1766.97	516.63	223.98	21.52	8.53

observations in Scenario 1, mainly due to the increased difficulty of solving subproblems as the problem size grows. Overall, although LBBD2 has similar Gap values to LBBD3, it takes longer to compute. Therefore, LBBD3 not only provides better solution quality but also offers significant computational efficiency.

The computational results for Scenario 3 (Tables 12–14, Fig. 5) indicate that, out of all 432 test instances, all methods except CP

are able to find the optimal solution. In terms of computational time, LBBD1 requires more time compared to LBBD2 and LBBD3, while the performance of LBBD2 and LBBD3 is nearly identical, with both methods achieving the best performance. The lack of a significant time advantage for LBBD3 is closely related to the specific characteristics of Scenario 3. In this scenario, the budget is relatively limited, while the processing time windows are more flexible. Due to the limited budget,

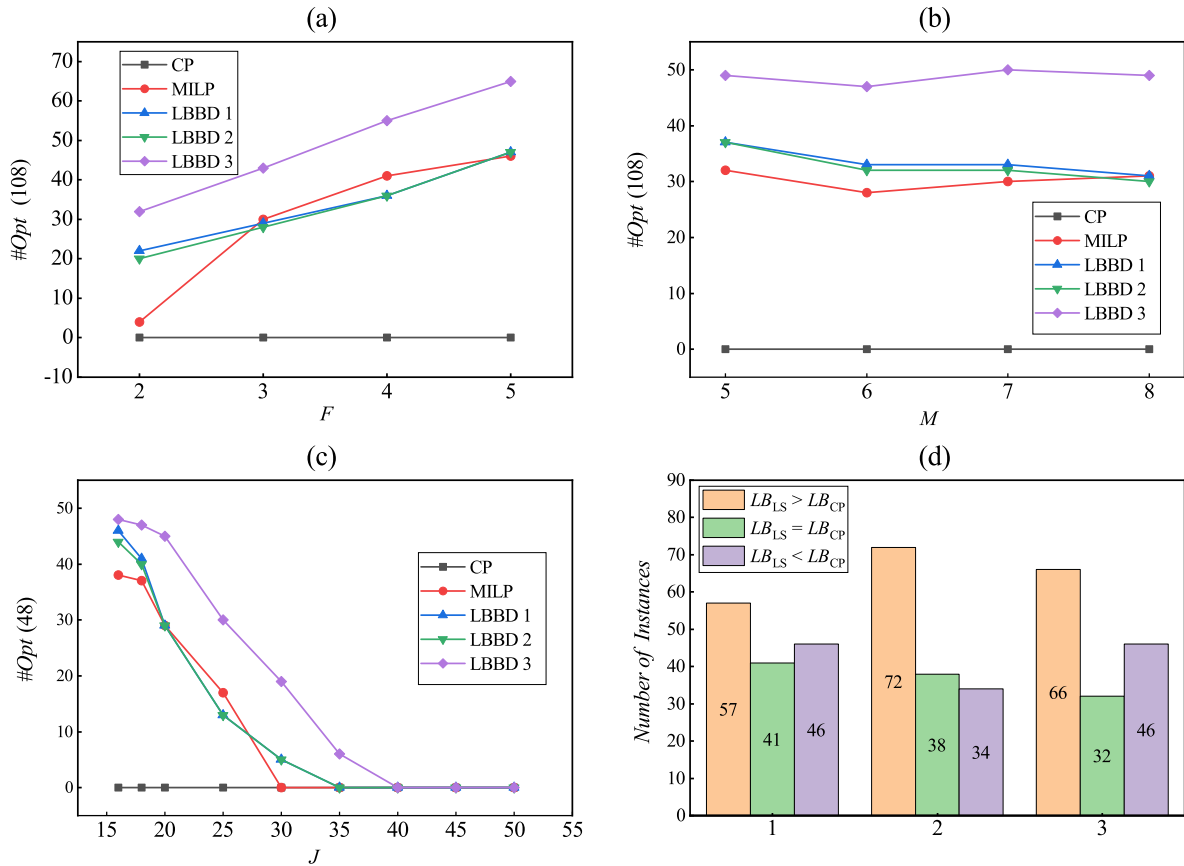


Fig. 3. The number of instances solved to optimality vs. problem sizes (Scenario 1). (a) The number of factories. (b) The number of machines. (c) The number of orders. (d) Statistics results of lower bounds provided by CP and LS across 144 instances. Note: #Opt represents the number of instances solved to optimality.

Table 11
Computational results of five methods across different factory sizes (Scenario 2).

F	Gap (%)					Time (s)				
	CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
2	1.91	0.18	0.28	0.00	0.00	1716.97	396.20	327.88	6.45	3.14
3	3.84	0.42	0.50	0.00	0.00	1783.64	552.34	330.96	19.33	13.32
4	6.52	0.88	0.32	0.00	0.00	1766.98	586.42	186.62	21.12	6.67
5	7.73	0.78	0.01	0.01	0.00	1800.29	531.58	50.47	39.19	10.98
Average	5.00	0.56	0.28	0.002	0.00	1766.97	516.63	223.98	21.52	8.53

Table 12
Computational results of five methods across different order sizes (Scenario 3).

J	Gap (%)					Time (s)				
	CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
16	4.15	0.00	0.00	0.00	0.00	1163.56	0.93	0.46	0.80	0.90
18	4.89	0.00	0.00	0.00	0.00	1762.75	1.23	0.41	0.66	0.82
20	3.84	0.00	0.00	0.00	0.00	1800.16	1.45	0.53	0.80	0.90
25	3.12	0.00	0.00	0.00	0.00	1800.17	2.24	0.50	0.60	0.62
30	2.24	0.00	0.00	0.00	0.00	1800.17	2.35	0.61	0.61	0.59
35	1.61	0.00	0.00	0.00	0.00	1800.18	3.18	0.79	0.61	0.62
40	1.53	0.00	0.00	0.00	0.00	1800.20	12.42	1.86	0.74	0.74
45	1.45	0.00	0.00	0.00	0.00	1762.76	6.16	1.22	0.65	0.69
50	1.09	0.00	0.00	0.00	0.00	1800.20	11.67	1.53	0.62	0.67
Average	2.66	0.00	0.00	0.00	0.00	1721.13	4.62	0.88	0.68	0.73

fewer orders are accepted in the main problem, and the wider time windows reduce the number of iterations required to reach the optimal solution. As a result, the advantages of domain search are not fully exploited in this scenario.

In this study, the performance of five algorithms is compared across different scenarios to assess their efficiency and effectiveness in solving the problem. Specifically, the best-performing algorithm in each scenario is as follows: In Scenario 1 (tight due dates and a high budget:

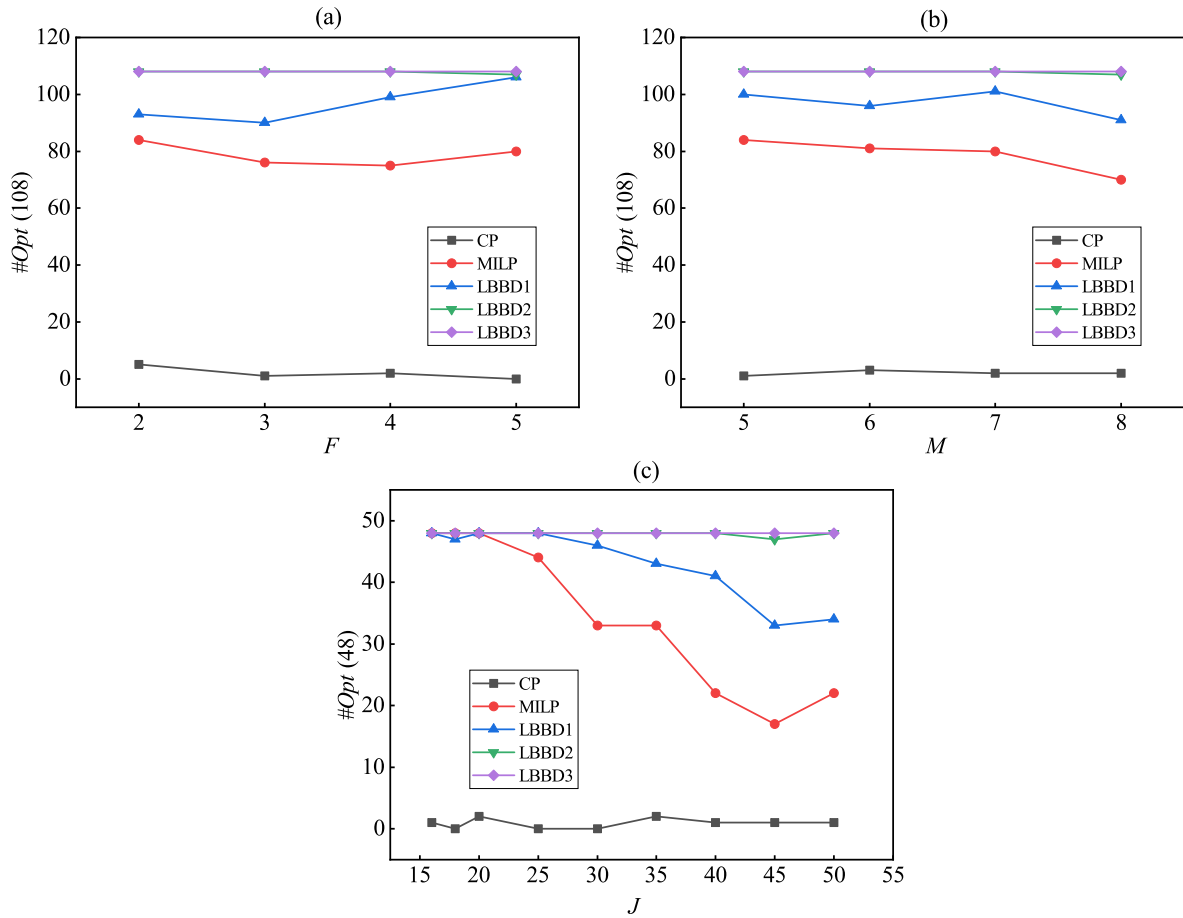


Fig. 4. The number of instances solved to optimality vs. problem sizes (Scenario 2). (a) The number of factories. (b) The number of machines. (c) The number of orders.

Table 13
Computational results of five methods across different machine sizes (Scenario 3).

M	Gap (%)					Time (s)				
	CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
5	2.21	0.00	0.00	0.00	0.00	1683.69	2.68	0.65	0.66	0.72
6	3.04	0.00	0.00	0.00	0.00	1750.24	5.04	0.80	0.74	0.85
7	2.74	0.00	0.00	0.00	0.00	1717.00	6.77	1.14	0.67	0.66
8	2.64	0.00	0.00	0.00	0.00	1733.59	6.77	0.93	0.64	0.68
Average	2.66	0.00	0.00	0.00	0.00	1721.13	4.62	0.88	0.68	0.73

Table 14
Computational results of five methods across different factory sizes (Scenario 3).

F	Gap (%)					Time (s)				
	CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
2	2.62	0.00	0.00	0.00	0.00	1716.96	2.69	0.54	0.48	0.53
3	2.42	0.00	0.00	0.00	0.00	1733.59	2.60	0.72	0.56	0.62
4	2.71	0.00	0.00	0.00	0.00	1700.31	6.98	1.22	0.82	0.86
5	2.88	0.00	0.00	0.00	0.00	1733.64	6.23	1.03	0.85	0.91
Average	2.66	0.00	0.00	0.00	0.00	1721.13	4.62	0.88	0.68	0.73

$\alpha = 0.3$, $\beta = 0.7$), LBBD3 performs the best, achieving the smallest Gap value and the shortest computation time. In Scenario 2 (moderate due dates and a balanced budget: $\alpha = 0.5$, $\beta = 0.5$), LBBD3 continues to lead, consistently finding the optimal solution and outperforming the other methods in both Gap and computation time. In Scenario 3 (relaxed due dates and a low budget: $\alpha = 0.7$, $\beta = 0.3$), all algorithms except CP successfully find the optimal solution, with LBBD2 and LBBD3 performing similarly. Overall, LBBD3 outperforms the other algorithms across all scenarios, particularly in terms of computational efficiency and solution quality.

5.4. Case study of engineering practice

To assess the applicability of the LBBD3 algorithm, it was implemented in an industrial case study focusing on the research and manufacturing processes of a large-scale automotive camera lens production company. Through an in-depth investigation, the manufacturing process of automotive camera lenses was found to comprise five key stages: spacer assembly, lens and infrared assembly, MTF testing, stray light testing, and appearance inspection. The production data were directly collected from the manufacturing site, encompassing four factories.

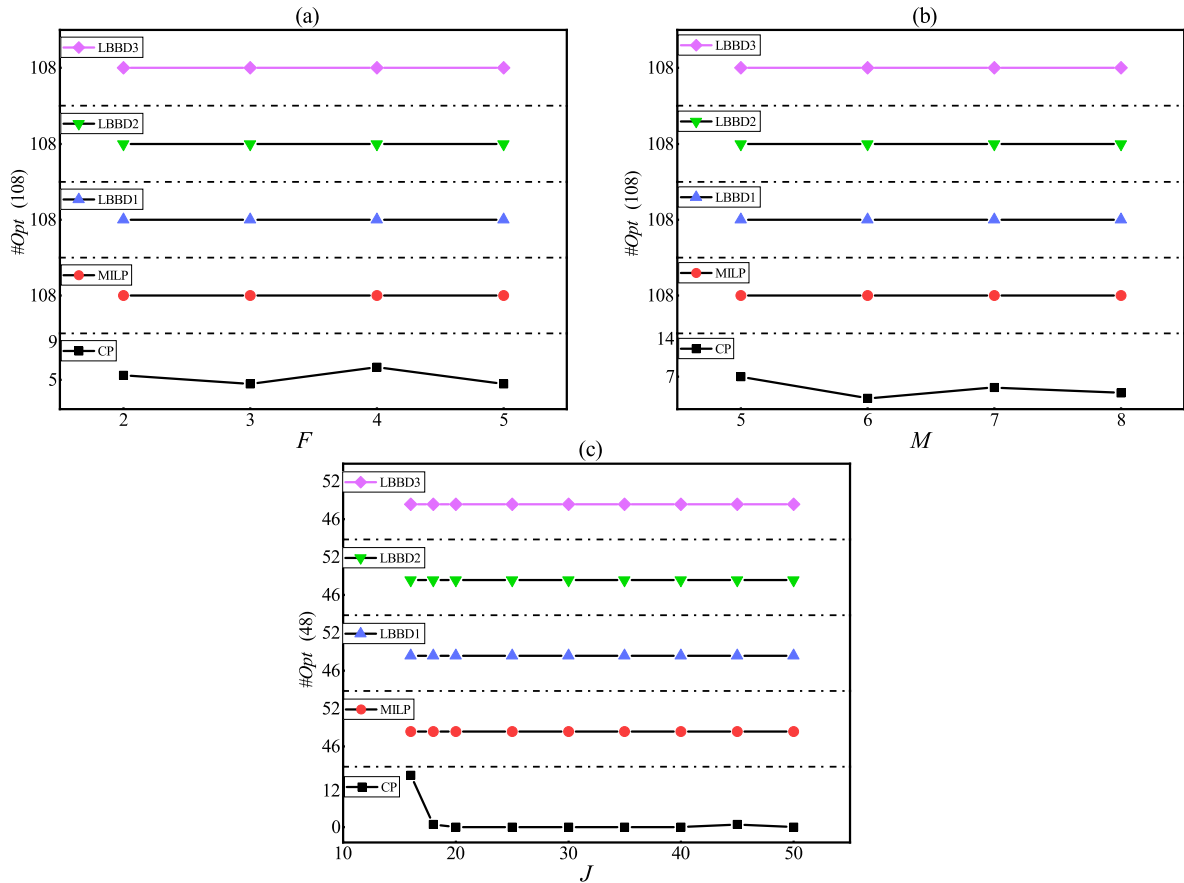


Fig. 5. The number of instances solved to optimality vs. problem sizes (Scenario 3). (a) The number of factories. (b) The number of machines. (c) The number of orders.

Table 15
The processing times of automotive camera lenses.

Order types	F_1					F_2					F_3					F_4				
	M_1	M_2	M_3	M_4	M_5	M_1	M_2	M_3	M_4	M_5	M_1	M_2	M_3	M_4	M_5	M_1	M_2	M_3	M_4	M_5
J_1	20	21	30	27	21	27	23	27	28	13	16	27	11	12	27	24	26	17	23	30
J_2	15	19	18	16	14	20	15	19	13	20	30	19	13	18	15	18	22	30	27	20
J_3	17	10	30	17	14	16	30	11	16	22	29	26	17	28	30	22	13	26	22	30
J_4	30	10	20	22	18	28	29	12	15	15	25	28	28	27	24	20	14	21	22	13
J_5	29	28	15	19	15	16	28	29	22	24	16	21	10	17	20	27	24	15	15	12
J_6	16	28	22	22	26	20	29	11	30	15	20	24	13	28	19	20	11	26	12	30
J_7	21	14	12	23	10	25	13	24	30	16	19	21	10	19	24	27	15	14	20	15
J_8	29	27	16	26	12	22	22	13	20	18	25	12	22	24	29	29	14	30	14	24

Table 16
The processing information of automotive camera lenses.

Order types	Processing information						
	V				$[r,d]$	R	E
	F_1	F_2	F_3	F_4			
J_1	12	18	21	7	[19,211]	130	[1,0,0,0]
J_2	22	16	14	6	[5,208]	133	[0,1,1,1]
J_3	24	16	8	13	[5,211]	107	[1,0,1,0]
J_4	11	16	5	21	[20,202]	115	[0,1,0,0]
J_5	12	8	25	16	[20,207]	150	[1,1,0,0]
J_6	10	11	16	25	[11,215]	136	[1,1,1,1]
J_7	23	9	14	20	[5,220]	131	[1,0,1,0]
J_8	19	20	5	12	[7,226]	133	[0,1,1,1]

Table 17
The optimization results of automotive camera lens manufacturing scheduling.

Methods	Profit UB	Profit LB	Budget used	Accepted orders	Profit growth
MILP	3247	2308	90.10%	18	
LBBBD3	2765	2421	96.60%	19	4.90%

Since the OAS_EAB_DHNPFSF problem is a feasibility problem, the obtained UB is inherently infeasible. A smaller UB indicates a solution that is closer to optimality. In contrast, the LB is both feasible and attainable. As presented in Table 17, the MILP method achieves a profit of 2308, with 18 accepted orders and a budget utilization rate of 90.10%. By implementing the LBBBD3 algorithm, the profit increases to 2421, the number of accepted orders rises to 19, corresponding to a profit growth rate of 4.9%, while the budget utilization improves to 96.60%. Furthermore, the substantial reduction in the UB suggests that the obtained solution is now closer to the optimal schedule. These findings highlight the strong effectiveness of the LBBBD3 algorithm

A total of eight types of automotive camera lenses were analyzed, with each type consisting of five batches, leading to a total of 40 orders. Detailed processing times and additional relevant information are presented in Tables 15–16.

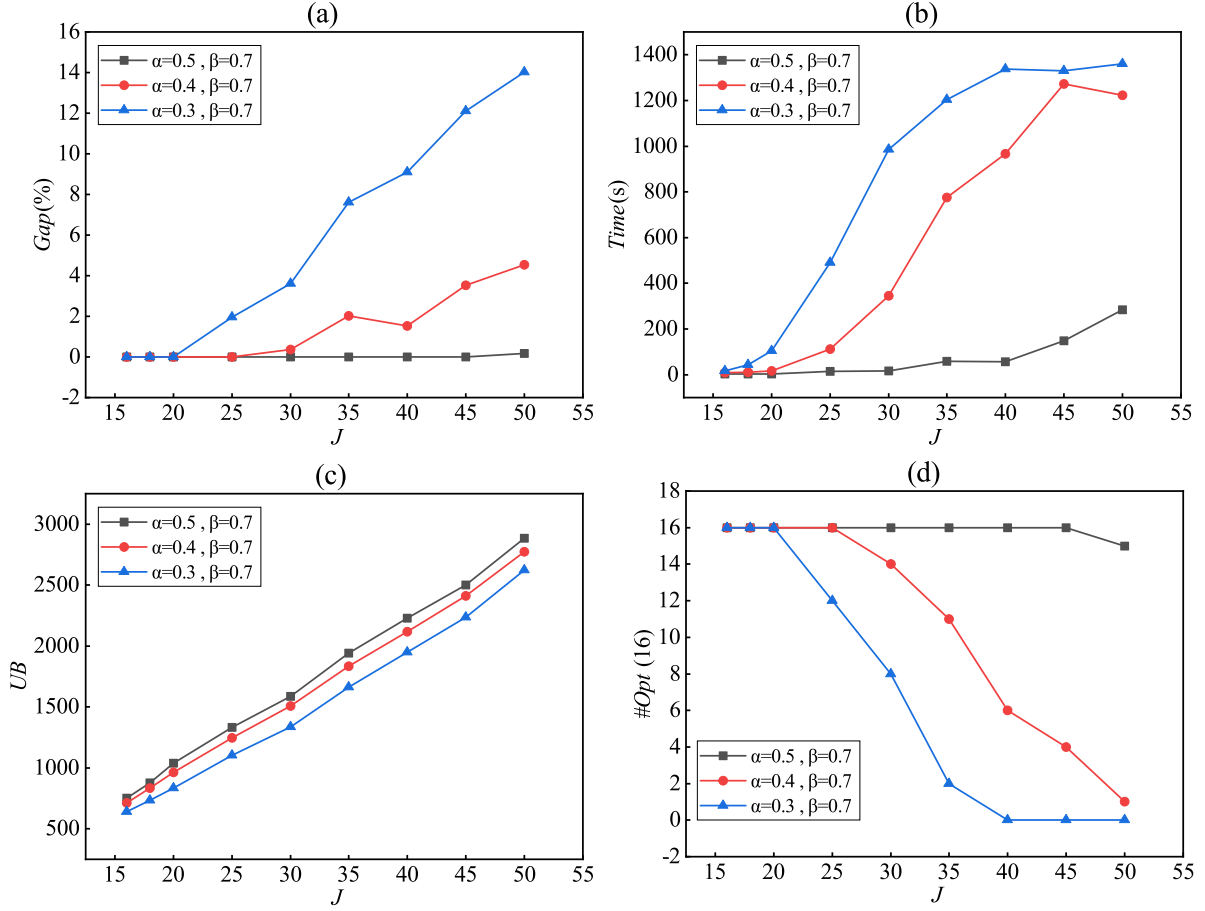


Fig. 6. Sensitivity analysis on due date: Gap, solution time, UB and #Opt vs. J . (a) Gap (b) solution time. (c) UB. (d) #Opt.

in addressing real-world industrial challenges, thereby significantly enhancing economic efficiency.

5.5. Sensitivity analysis

In this section, sensitivity analysis is conducted by varying three key factors: the budget, the due date, and the problem size. The analysis is carried out through three different experimental setups. First, the budget coefficient is fixed at $\beta = 0.7$, and the due date coefficient α is varied across three values: 0.5, 0.4, and 0.3. Second, the due date coefficient is fixed at $\alpha = 0.5$, and the budget coefficient β is adjusted to the values 0.7, 0.5, and 0.3. Finally, the impact of varying the problem size, including the number of machines (M), order size (J), and the number of factories (F), is analyzed. All instances are solved using the CP_LBBD_{CP_LS} method, and the results are presented in Figs. 6–8.

5.5.1. Effect of due date

In the sensitivity analysis where the budget coefficient ($\beta = 0.7$) is fixed and the due date coefficient (α) is varied, four key parameters — Gap, Time, UB, and #Opt — demonstrate specific trends as the due date coefficient decreases. As shown in Fig. 6(a), the Gap value increases significantly as α decreases, particularly for larger order sizes. For instance, when $J = 50$, the gap increases from 0% to 14.02%, suggesting that tighter due dates result in a significant deterioration in solution quality. Moreover, as shown in Fig. 6(b), the computation time increases substantially as α decreases, suggesting that more urgent due dates complicate the problem-solving process. While the maximum possible profit does not exhibit significant changes for smaller instances, Fig. 6(c) indicates that as α decreases, the maximum profit gradually decreases, suggesting that tighter due dates reduce the achievable

profit. Finally, as shown in Fig. 6(d), the number of optimal solutions decreases as α decreases, especially for larger order sizes, implying that tighter due dates make it more difficult for the algorithm to find the optimal solution.

5.5.2. Effect of budget

In the sensitivity analysis where the due date coefficient ($\alpha = 0.3$) is fixed and the budget coefficient (β) is varied, it can be observed from Fig. 7 that as the budget coefficient β decreases, both the solution quality (Gap, #Opt) and the solution time improve. However, tighter budget conditions significantly impact the maximum possible profit (UB), leading to a decrease in its value.

5.5.3. Effect of problem size

The sensitivity analysis on problem size is presented in Fig. 8. As shown in Fig. 8(a), with an increase in order size (J), both the Gap and solution time increase, indicating that larger order sizes lead to greater computational difficulty. In Fig. 8(b), when the number of machines increases from 5 to 8, the Gap and solution time show minimal changes, suggesting that the impact of the number of machines on the problem is relatively small. Fig. 8(c) reveals that increasing the number of factories actually makes the problem easier to solve, contrary to theoretical expectations. To explore the specific influence factors, the average number of orders per factory (J/F) is analyzed. As shown in Fig. 8(d), when $J/F \leq 5$, the Gap is 0, and as J/F increases, both the Gap and solution time increase significantly. This indicates that for this problem, the number of factories has a smaller effect on the problem compared to the number of orders assigned to each factory.

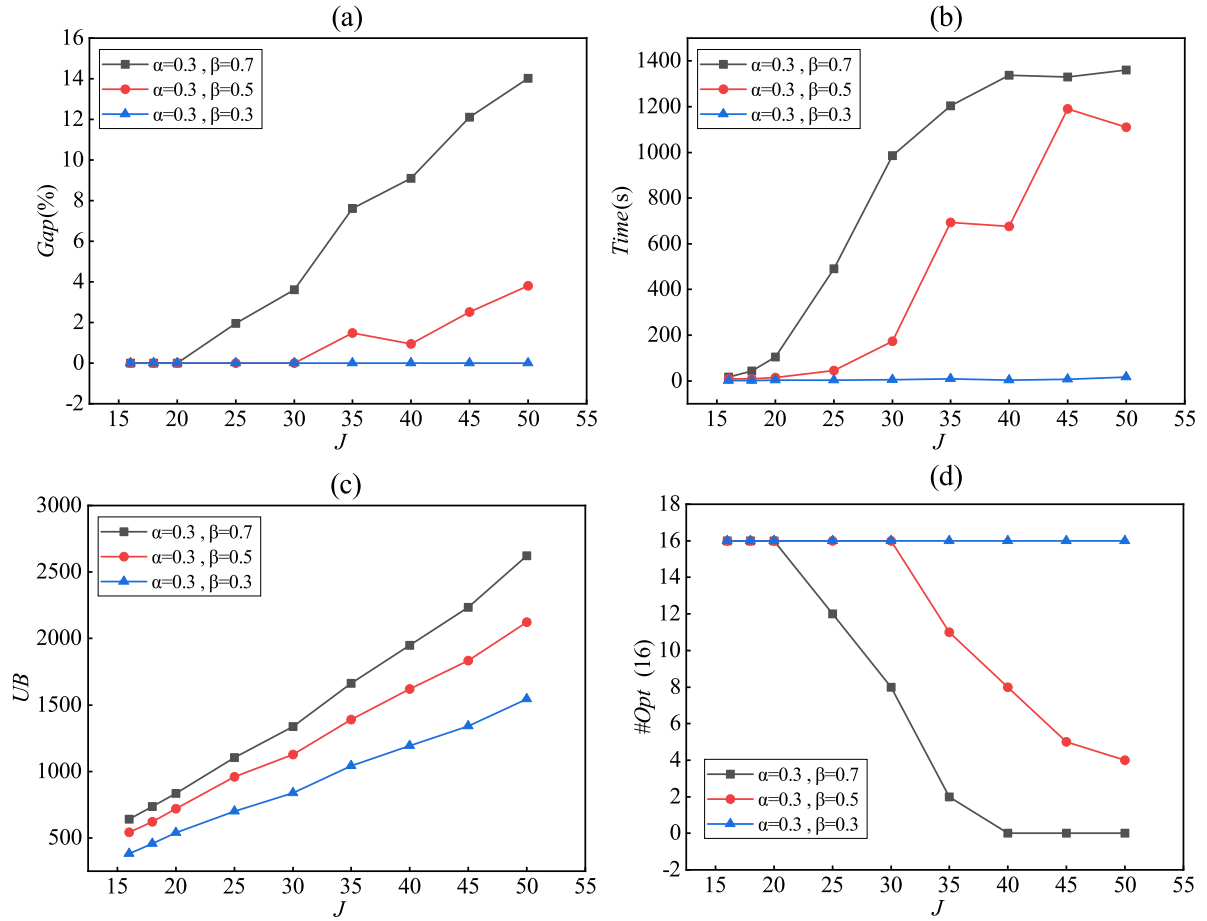


Fig. 7. Sensitivity analysis on budget: Gap , solution time, UB and $\#Opt$ vs. J . (a) Gap (b) solution time. (c) UB . (d) $\#Opt$.

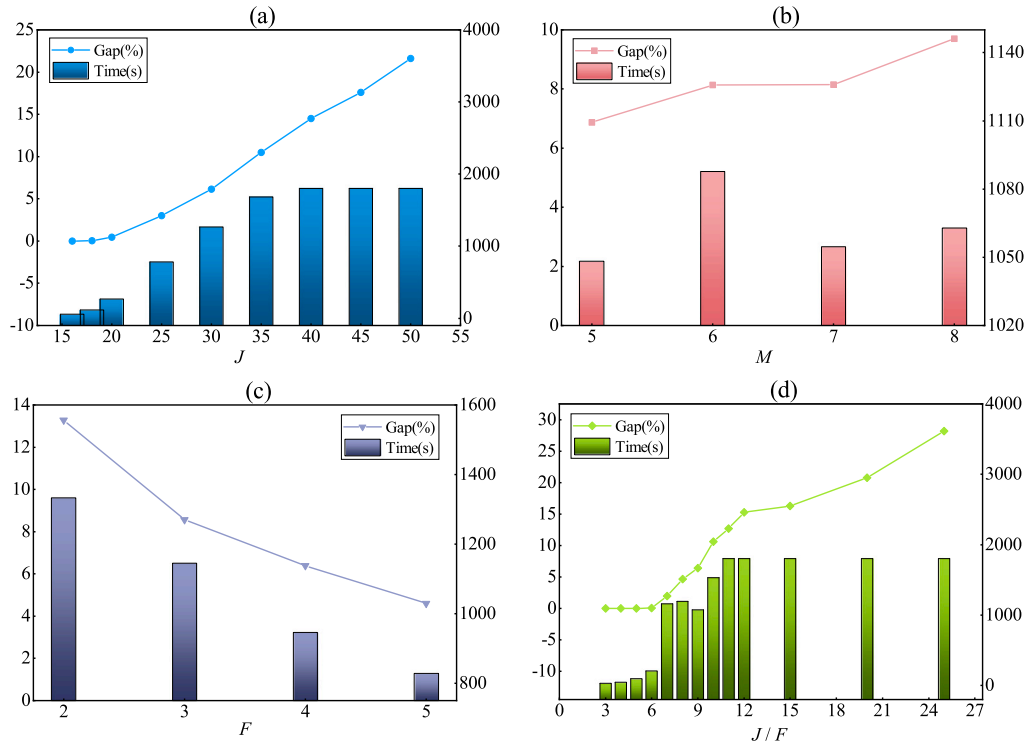


Fig. 8. Solution time and gap vs. problem sizes of LBBD3. (a) The number of orders. (b) The number of machines. (c) The number of factories. (d) The number of the ratio of F to J .

6. Conclusions and future research

This study addresses the distributed non-permutation flow shop scheduling problem with budget constraints, order-to-factory eligibility, processing costs, and processing time availability, with the goal of maximizing net profit while satisfying these constraints. First, highlighted by Naderi et al. (2023), a Manne-based MILP model and a CP model are formulated. Leveraging the decomposable structure of the OAS_EAB_DHNPFS problem, three LBB algorithm are proposed: CP_LBB_{MILP}, CP_LBB_{CP} and CP_LBB_{CP_LS}, in which the original problem is decomposed into an AMP and a set of SSPs and solved alternately. In these LBB approaches, since the subproblems are feasibility problems rather than optimization problems, the lower bound obtained from CP after t_{max}^{CP} is used as the LBB lower bound. Additionally, in the CP_LBB_{CP_LS} algorithm, a LS is further designed to filter cuts and improve the lower bound, thus increasing the potential for reducing the gap within a limited time. The experimental results show that, in small-size instances, considering both solution time and quality, all three LBB variants outperform MILP and CP. In large-size instances, across the three scenarios, the LBB approach combined with LS yields the best performance. These findings underscore the effectiveness of integrating LS and LBB in solving the OAS_EAB_DHNPFS problem. The significance of this study lies in two key aspects. Theoretically, the problem explored in this paper constitutes a substantial extension of traditional OAS and DPFS models. By incorporating constraints that better reflect real-world production conditions, this research offers a more practical framework for scheduling problems. This extension not only strengthens the theoretical foundation of scheduling theory but also bridges the gap between abstract models and industrial applications, providing a more precise representation of the complexities in modern manufacturing systems.

From a practical application perspective, this study provides valuable managerial insights for manufacturers through sensitivity analysis and real-world case applications. Manufacturers must balance budget flexibility and deadline constraints to optimize scheduling, as higher budgets and tighter deadlines increase scheduling complexity. Even the allocation of orders among different factories plays a crucial role in reducing inefficiencies and enhancing overall productivity. Effective management of these factors can improve production planning, resource utilization, and operational adaptability. Moreover, the development of adaptive scheduling systems that not only optimize production processes but also accommodate unforeseen demand fluctuations is essential for enhancing manufacturing resilience. This research holds practical significance for industries such as automotive, electronics, and consumer goods manufacturing, contributing to increased corporate profitability and fostering innovation in production planning and scheduling.

Future research can be conducted on the following aspects: (1) Extend LBB methods to other distributed manufacturing scheduling problems, such as distributed job shop and open shop scheduling. (2) Incorporate real-world constraints like maintenance scheduling, supply chain disruptions, and resource sharing between production lines into the LBB approach. (3) Investigate the impact of uncertainty, such as processing time variations or machine breakdowns, and integrate these factors into the LBB framework for improved robustness and adaptability. (4) Explore the use of multi-agent systems to distribute scheduling tasks across factories and machines, enhancing flexibility and scalability.

CRediT authorship contribution statement

Fuli Xiong: Conceptualization, Methodology, Formal analysis, Investigation, Data curation, Writing-Original Draft, Writing – review & editing, Software, Supervision, Resources, Funding acquisition, Project administration. **Lin Jing:** Formal analysis, Methodology, Software, Writing – original draft, Writing – review & editing. **Chengfei Xi-** **ang:** Software, Validation, Data curation, Visualization. **Yitong Han:** Investigation, Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported in part by the National Natural Science Foundation of China [grant number 61473216] and the Natural Science Basic Research Program of Shaanxi Province, China [grant number 2023-JC-YB-582].

Appendix. Detail results of large-size instances and algorithm performance

In this section, we present the detailed data for large-size instances, including computational results, performance comparisons, and the evaluation of different algorithms. These data provide support for the validation and discussion of the key research findings (see Tables 18–20).

Data availability

Data will be made available on request.

Table 18
Computational results for large-size instances (Scenario 1).

F	M	J	Gap (%)					Time (s)				
			CP	MILP	LBB1	LBB2	LBB3	CP	MILP	LBB1	LBB2	LBB3
2	(5,6,7,8)	16	20.89	7.85	0.91	1.39	0.00	1800.16	1432.94	653.66	788.84	106.63
		20	24.27	24.01	7.83	8.35	1.89	1800.11	1795.20	1596.25	1578.86	744.94
		25	24.11	26.05	17.68	17.56	9.95	1800.12	1800.11	1800.00	1800.00	1800.00
		30	22.24	25.36	19.24	18.41	13.17	1800.11	1800.13	1800.00	1800.00	1800.00
		35	27.84	32.11	28.27	25.68	21.08	1800.15	1800.16	1800.00	1800.00	1800.00
		40	24.38	29.91	28.25	25.63	20.78	1800.10	1800.22	1800.00	1800.00	1800.00
		45	26.29	32.16	29.13	26.93	24.32	1800.15	1800.30	1800.00	1800.00	1800.00
		50	28.31	37.06	31.73	29.86	28.16	1800.16	1800.38	1800.00	1800.00	1800.00
3	(5,6,7,8)	16	29.94	0.00	0.31	0.56	0.00	1800.15	50.68	413.10	542.66	64.73
		18	26.49	0.00	0.88	1.26	0.00	1800.16	145.26	529.36	691.44	39.39
		20	24.58	2.49	2.25	2.32	0.00	1800.16	1142.75	827.35	1100.37	137.52
		25	26.97	21.94	13.77	14.07	2.12	1800.14	1800.25	1800.00	1800.00	1080.13
		30	28.44	28.72	21.25	21.37	9.38	1800.16	1800.27	1800.00	1800.00	1786.85

(continued on next page)

Table 18 (continued).

F	M	J	Gap (%)					Time (s)				
			CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
4	(5,6,7,8)	35	25.58	27.05	20.49	20.67	11.33	1800.14	1800.29	1800.00	1800.00	1800.00
		40	24.36	26.72	21.13	21.01	14.77	1800.24	1800.33	1800.00	1800.00	1800.00
		45	28.31	31.11	26.23	25.82	19.42	1800.15	1800.37	1800.00	1800.00	1800.00
		50	25.06	29.61	25.65	24.69	20.12	1800.14	1800.44	1800.00	1800.00	1800.00
		16	30.86	0.00	0.00	0.00	0.00	1800.16	11.44	81.23	169.61	32.33
		18	32.85	0.00	0.17	0.51	0.00	1800.16	35.43	287.58	381.78	35.99
		20	31.10	0.00	4.88	5.02	0.00	1800.15	454.74	720.18	750.19	121.31
		25	27.93	2.56	7.17	7.34	0.00	1800.12	1192.17	1190.36	1214.70	144.33
		30	26.38	14.52	11.53	11.66	1.83	1800.21	1800.27	1800.00	1800.00	983.07
		35	27.38	25.88	17.70	17.89	7.53	1800.20	1800.26	1800.00	1800.00	1800.00
		40	30.55	30.53	23.35	23.44	13.53	1800.15	1800.37	1800.00	1800.00	1800.00
		45	26.96	27.87	22.44	22.63	14.03	1800.19	1800.36	1800.00	1800.00	1800.00
5	(5,6,7,8)	50	30.29	32.66	28.68	28.68	20.60	1800.16	1800.51	1800.00	1800.00	1800.00
		16	36.61	0.00	0.00	0.00	0.00	1800.14	8.77	36.05	105.72	28.52
		18	34.87	0.00	0.00	0.00	0.00	1800.18	20.69	52.66	134.77	30.22
		20	35.05	0.00	1.19	1.30	0.00	1800.17	72.46	508.23	708.09	57.01
		25	29.36	0.50	1.45	1.86	0.00	1800.17	532.54	883.11	971.01	100.65
		30	29.67	9.25	6.99	7.37	0.21	1800.14	1800.24	1401.06	1456.41	503.40
		35	27.07	15.05	14.18	14.48	2.08	1800.22	1800.27	1800.00	1800.00	1337.67
		40	29.73	26.02	19.05	19.23	8.97	1800.23	1800.30	1800.00	1800.00	1800.00
		45	29.54	31.58	21.78	21.66	12.59	1800.21	1800.42	1800.00	1800.00	1800.00
		50	31.23	34.02	26.17	26.13	17.56	1800.23	1800.39	1800.00	1800.00	1800.00

Table 19

Computational results for large-size instances (Scenario 2).

F	M	J	Gap (%)					Time (s)				
			CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
2	(5,6,7,8)	16	3.06	0.00	0.00	0.00	0.00	1650.20	7.29	1.51	1.65	3.18
		18	2.67	0.00	0.00	0.00	0.00	1800.25	10.79	1.80	0.92	0.88
		20	2.92	0.00	0.00	0.00	0.00	1650.65	3.19	1.82	0.53	0.56
		25	1.92	0.01	0.00	0.00	0.00	1800.22	203.61	89.14	0.74	0.89
		30	1.63	0.22	0.68	0.00	0.00	1800.28	606.38	449.84	1.17	1.12
		35	1.38	0.30	0.34	0.00	0.00	1650.34	603.53	503.47	1.57	2.26
		40	1.28	0.25	0.51	0.00	0.00	1800.20	318.39	292.81	2.88	5.31
		45	1.01	0.30	0.51	0.00	0.00	1650.33	905.45	867.72	3.44	2.26
		50	1.32	0.53	0.48	0.00	0.00	1650.26	907.16	742.84	45.14	11.81
3	(5,6,7,8)	16	8.41	0.00	0.00	0.00	0.00	1800.31	56.72	4.39	5.04	2.88
		18	5.76	0.00	0.00	0.00	0.00	1800.28	15.85	2.51	5.42	2.59
		20	4.61	0.00	0.00	0.00	0.00	1650.60	45.81	1.59	1.94	1.34
		25	3.23	0.00	0.00	0.00	0.00	1800.27	73.16	5.28	2.68	1.97
		30	3.72	1.12	0.00	0.00	0.00	1800.23	886.66	61.87	14.83	4.38
		35	2.46	0.85	0.40	0.00	0.00	1800.30	854.08	499.24	6.27	5.11
		40	1.55	0.69	0.66	0.00	0.00	1800.22	903.62	745.79	22.68	17.45
		45	2.38	0.55	1.66	0.00	0.00	1800.24	1289.46	1038.19	112.05	54.28
		50	2.41	0.54	1.74	0.00	0.00	1800.30	845.70	619.77	3.06	29.86
4	(5,6,7,8)	16	12.57	0.00	0.00	0.00	0.00	1800.31	3.80	2.03	5.46	2.93
		18	12.94	0.00	0.00	0.00	0.00	1800.26	15.38	4.87	8.64	4.60
		20	7.72	0.00	0.00	0.00	0.00	1800.32	14.63	3.88	7.52	3.62
		25	6.78	0.39	0.00	0.00	0.00	1800.26	321.77	8.06	14.86	4.02
		30	3.90	1.67	0.00	0.00	0.00	1800.31	660.29	29.61	35.49	9.73
		35	3.88	0.89	0.00	0.00	0.00	1650.32	689.25	14.84	8.80	4.06
		40	3.85	2.63	0.00	0.00	0.00	1650.33	1362.67	224.25	78.37	14.70
		45	4.07	1.20	1.06	0.00	0.00	1800.30	954.81	409.46	24.19	7.34
		50	2.93	1.12	1.83	0.00	0.00	1800.43	1255.19	982.53	6.76	9.02
5	(5,6,7,8)	16	16.44	0.00	0.00	0.00	0.00	1800.26	3.44	3.09	6.41	3.90
		18	13.46	0.00	0.00	0.00	0.00	1800.30	4.61	3.82	8.85	4.10
		20	13.70	0.00	0.00	0.00	0.00	1800.23	11.69	17.88	19.16	6.63
		25	7.11	0.00	0.00	0.00	0.00	1800.19	30.67	11.98	29.79	6.23
		30	5.26	0.43	0.00	0.00	0.00	1800.30	409.90	8.25	13.24	7.67
		35	3.35	0.25	0.00	0.00	0.00	1800.24	351.47	6.59	9.47	3.41
		40	3.26	1.67	0.00	0.00	0.00	1800.26	1385.40	25.04	25.87	6.55
		45	3.40	2.44	0.10	0.08	0.00	1800.31	1376.97	212.98	200.99	47.49
		50	3.57	2.26	0.00	0.00	0.00	1800.50	1210.04	164.60	38.95	12.86

Table 20
Computational results for large-size instances (Scenario 3).

F	M	J	Gap (%)					Time (s)				
			CP	MILP	LBBD1	LBBD2	LBBD3	CP	MILP	LBBD1	LBBD2	LBBD3
2	(5,6,7,8)	16	3.51	0.00	0.00	0.00	0.00	1201.15	7.29	0.30	0.56	0.44
		18	4.76	0.00	0.00	0.00	0.00	1800.13	10.79	0.32	0.38	0.49
		20	4.23	0.00	0.00	0.00	0.00	1800.16	3.19	0.43	0.42	0.50
		25	3.24	0.01	0.00	0.00	0.00	1800.13	203.61	0.39	0.42	0.49
		30	2.03	0.22	0.00	0.00	0.00	1800.12	606.38	0.48	0.47	0.49
		35	1.67	0.30	0.00	0.00	0.00	1800.19	603.53	0.57	0.47	0.52
		40	1.55	0.25	0.00	0.00	0.00	1800.15	318.39	0.71	0.52	0.61
		45	1.71	0.30	0.00	0.00	0.00	1650.48	905.45	0.78	0.58	0.66
		50	0.88	0.53	0.00	0.00	0.00	1800.16	907.16	0.88	0.46	0.53
3	(5,6,7,8)	16	3.66	0.00	0.00	0.00	0.00	1200.96	56.72	0.35	0.64	0.68
		18	5.23	0.00	0.00	0.00	0.00	1800.11	15.85	0.38	0.55	0.60
		20	3.27	0.00	0.00	0.00	0.00	1800.18	45.81	0.47	0.52	0.64
		25	2.11	0.00	0.00	0.00	0.00	1800.20	73.16	0.44	0.51	0.56
		30	2.53	1.12	0.00	0.00	0.00	1800.21	886.66	0.64	0.62	0.60
		35	1.34	0.85	0.00	0.00	0.00	1800.19	854.08	0.78	0.49	0.58
		40	1.46	0.69	0.00	0.00	0.00	1800.16	903.62	0.89	0.61	0.58
		45	1.07	0.55	0.00	0.00	0.00	1800.15	1289.46	1.18	0.54	0.65
		50	1.14	0.54	0.00	0.00	0.00	1800.19	845.70	1.32	0.60	0.68
4	(5,6,7,8)	16	2.46	0.00	0.00	0.00	0.00	901.34	3.80	0.52	0.89	1.12
		18	5.18	0.00	0.00	0.00	0.00	1800.16	15.38	0.57	1.07	1.44
		20	3.75	0.00	0.00	0.00	0.00	1800.15	14.63	0.47	0.83	0.88
		25	4.45	0.39	0.00	0.00	0.00	1800.16	321.77	0.61	0.71	0.73
		30	2.20	1.67	0.00	0.00	0.00	1800.17	660.29	0.58	0.65	0.59
		35	1.92	0.89	0.00	0.00	0.00	1800.15	689.25	0.86	0.76	0.67
		40	1.45	2.63	0.00	0.00	0.00	1800.25	1362.67	4.58	1.08	0.95
		45	1.87	1.20	0.00	0.00	0.00	1800.20	954.81	1.32	0.72	0.67
		50	1.08	1.12	0.00	0.00	0.00	1800.22	1255.19	1.50	0.69	0.68
5	(5,6,7,8)	16	6.98	0.00	0.00	0.00	0.00	1350.81	3.44	0.66	1.10	1.36
		18	4.38	0.00	0.00	0.00	0.00	1650.61	4.61	0.38	0.66	0.76
		20	4.10	0.00	0.00	0.00	0.00	1800.14	11.69	0.73	1.45	1.58
		25	2.67	0.00	0.00	0.00	0.00	1800.19	30.67	0.57	0.76	0.71
		30	2.19	0.43	0.00	0.00	0.00	1800.19	409.90	0.74	0.71	0.68
		35	1.51	0.25	0.00	0.00	0.00	1800.21	351.47	0.97	0.72	0.71
		40	1.64	1.67	0.00	0.00	0.00	1800.22	1385.40	1.24	0.77	0.81
		45	1.14	2.44	0.00	0.00	0.00	1800.20	1376.97	1.59	0.76	0.78
		50	1.26	2.26	0.00	0.00	0.00	1800.22	1210.04	2.41	0.73	0.80

References

Abbaas, O., & Ventura, J. A. (2024). A multi-agent resource bidding algorithm for order acceptance and assembly job shop scheduling. *International Journal of Production Research*, 62(13), 4856–4883.

Ahmed, M., & Younès, B. (2021). An effective iterated greedy algorithm for blocking hybrid flow shop problem with due date window. *RAIRO - Operations Research*, 55(3), 1603–1616.

Bai, D., Liu, T., Zhang, Y., Chu, F., Qin, H., Gao, L., et al. (2024). Scheduling a distributed permutation flowshop with uniform machines and release dates. *IEEE Transactions on Automation Science and Engineering*, 1–13.

Bartal, Y., Leonardi, S., Marchetti-Spaccamela, A., Sgall, J., & Stougie, L. (2000). Multiprocessor scheduling with rejection. *SIAM Journal on Discrete Mathematics*, 13(1), 64–78.

Benders, J. F. (1962). Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik*, 4(1), 238–252.

Bin, S. H., & Bo, W. J. (2020). Research on common due window assignment flowshop scheduling with learning effect and resource allocation. *Engineering Optimization*, 52(4), 669–686.

Boland, N., Savelsbergh, M., & Waterer, H. (2012). A general benders decomposition framework for solving nonlinear and mixed-integer nonlinear programming problems. *Computational Optimization and Applications*, 51(2), 361–391.

Carla, J., Laurent, H., & Pierre, L. (2023). Logic-based benders decomposition for the preemptive flexible job-shop scheduling problem. *Computers & Operations Research*, 152.

Chen, J., Ye, X., Ma, W., & Xu, D. (2024). Logic-based benders decomposition for order acceptance and scheduling on heterogeneous factories with carbon caps. *Computers & Operations Research*, 168, Article 106706.

Cordeau, J.-F., Gendron, B., & Laporte, G. (2017). A benders decomposition approach for a multi-vehicle routing problem. *Transportation Science*, 45(3), 401–414.

Feng, X., Zhao, F., Jiang, G., Tao, T., & Mei, X. (2024). A tabu memory based iterated greedy algorithm for the distributed heterogeneous permutation flowshop scheduling problem with the total tardiness criterion. *Expert Systems with Applications*, 238(PD).

Fuli, X., & Ziyang, Y. (2024). Multi-thread parallel iterated greedy for flow shop order acceptance and scheduling problem. *Computer Integrated Manufacturing System*, 30(11), 3918.

Gogos, C. (2023). Solving the distributed permutation flow-shop scheduling problem using constrained programming. *Applied Sciences*, 13(23).

Han, B., Pan, Q.-K., & Gao, L. (2023). A cooperative iterated greedy algorithm for the serial distributed permutation flowshop scheduling problem. *International Journal of Production Research*, 62(12), 4245–4272.

Han, Z., Shi, H., & Yao, L. (2014). Time-window-based combined objective DE algorithm for hybrid flow-shop. *International Journal of Modelling, Identification and Control*, 21(3), 295–305.

Harjunkoski, I., & Grossmann, I. E. (2002). Decomposition techniques for multistage scheduling problems using mixed-integer and constraint programming methods. *Computers and Chemical Engineering*, 26(11), 1533–1552.

Hooker, J., & Ottosson, G. (2003). Logic-based benders decomposition. *Mathematical Programming*, 96(1), 33–60.

Hooker, J., Ottosson, G., Thorsteinsson, E., & Kim, H. (1999). On integrating constraint propagation and linear programming for combinatorial optimization. *Proceedings of the AAAI/IAAI Conference*, 136–142.

Hou, Y., Fu, Y., Gao, K., Zhang, H., & Ali, S. (2022). Modelling and optimization of integrated distributed flow shop scheduling and distribution problems with time windows. *Expert Systems with Applications*, 187.

Huang, K., Li, R., Gong, W., Wang, R., & Wei, H. (2023). BRCE: bi-roles co-evolution for energy-efficient distributed heterogeneous permutation flow shop scheduling with flexible machine speed. *Complex & Intelligent Systems*, 9(5), 4805–4816.

Huang, Y., Solomon, M., & Golden, B. (2020). A benders decomposition-based heuristic for the multi-objective maritime inventory routing problem. *European Journal of Operational Research*, 283(1), 53–67.

Jing, X.-L., Pan, Q.-K., & Gao, L. (2021). Local search-based metaheuristics for the robust distributed permutation flowshop problem. *Applied Soft Computing*, 105, Article 107247.

Jing, X.-L., Pan, Q.-K., Gao, L., & Wang, Y.-L. (2020). An effective iterated greedy algorithm for the distributed permutation flowshop scheduling with due windows. *Applied Soft Computing Journal*, 96.

Laporte, G., Vena, T., & Feliu, I. G. (2015). Recent advances in benders decomposition and its applications. *European Journal of Operational Research*, 242(1), 1–10.

- Lei, D., & Guo, X. (2015). A parallel neighborhood search for order acceptance and scheduling in flow shop environment. *International Journal of Production Economics*, 165, 12–18.
- Li, Y., Côté, J.-F., Coelho, L. C., Zhang, C., & Zhang, S. (2023). Order assignment and scheduling under processing and distribution time uncertainty. *European Journal of Operational Research*, 305(1), 148–163.
- Li, T., Li, J.-q., Chen, X.-l., & Li, J.-k. (2024). Solving distributed assembly blocking flowshop with order acceptance by knowledge-driven multiobjective algorithm. *Engineering Applications of Artificial Intelligence*, 137, Article 109220.
- Li, H., Li, X., & Gao, L. (2024). An iterated greedy algorithm with acceleration of job allocation probability for distributed heterogeneous permutation flowshop scheduling problem. *Swarm and Evolutionary Computation*, 88, Article 101580.
- Li, X., & Ventura, J. A. (2020). Exact algorithms for a joint order acceptance and scheduling problem. *International Journal of Production Economics*, 223.
- Li, R., Wang, L., Gong, W., & Ming, F. (2024). An evolutionary multitasking memetic algorithm for multi-objective distributed heterogeneous welding flow shop scheduling. *IEEE Transactions on Evolutionary Computation*.
- Lin, W.-C., Xu, J., Bai, D., Chung, I.-H., Liu, S.-C., & Wu, C.-C. (2019). Artificial bee colony algorithms for the order scheduling with release dates. *Soft Computing*, 23(18), 8677–8688.
- Liu, J., Han, Y., Wang, Y., Liu, Y., & Zhang, B. (2024). Distributed hybrid flowshop scheduling with consistent sublots under delivery time windows: A penalty lot-assisted iterated greedy algorithm. *Egyptian Informatics Journal*, 28, 100566–100566.
- Luo, H., Huang, G. Q., Zhang, Y. F., & Dai, Q. Y. (2011). Hybrid flowshop scheduling with batch-discrete processors and machine maintenance in time windows. *International Journal of Production Research*, 49(6), 1575–1603.
- Luo, L., & Yan, X. (2025). Scheduling of stochastic distributed hybrid flow-shop by hybrid estimation of distribution algorithm and proximal policy optimization. *Expert Systems with Applications*, Article 126523.
- Meng, T., & Pan, Q.-K. (2021). A distributed heterogeneous permutation flowshop scheduling problem with lot-streaming and carryover sequence-dependent setup time. *Swarm and Evolutionary Computation*, 60, Article 100804.
- Mou, J., Duan, P., Gao, L., Liu, X., & Li, J. (2022). An effective hybrid collaborative algorithm for energy-efficient distributed permutation flow-shop inverse scheduling. *Future Generation Computer Systems*, 128, 521–537.
- Naderi, B., & Ruiz, R. (2010). The distributed permutation flowshop scheduling problem. *Computers & Operations Research*, 37(4), 754–768.
- Naderi, B., Ruiz, R., & Roshanaei, V. (2023). Mixed-integer programming vs. Constraint programming for shop scheduling problems: New results and outlook. *INFORMS Journal on Computing*, 35(4), 817–843.
- Nobibon, F. T., & Leus, R. (2010). Exact algorithms for a generalization of the order acceptance and scheduling problem in a single-machine environment. *Computers & Operations Research*, 38(1), 367–378.
- Noroozi, A., Mazdeh, M. M., Heydari, M., & Rasti-Barzoki, M. (2018). Coordinating order acceptance and integrated production-distribution scheduling with batch delivery considering third party logistics distribution. *Journal of Manufacturing Systems*, 46, 29–45.
- Og, C., Salman, F. S., Yalçın, Z. B., et al. (2010). Order acceptance and scheduling decisions in make-to-order systems. *International Journal of Production Economics*, 125(1), 200–211.
- Ou, J., Zhong, X., & Wang, G. (2015). An improved heuristic for parallel machine scheduling with rejection. *European Journal of Operational Research*, 241(3), 653–661.
- Pan, Y., Gao, K., Li, Z., & Wu, N. (2023). Improved meta-heuristics for solving distributed lot-streaming permutation flow shop scheduling problems. *IEEE Transactions on Automation Science and Engineering*, 20(1), 361–371.
- Papadakos, N. (2008). Practical enhancements to the Magnanti–Wong method. *Operations Research Letters*, 36(4), 444–449.
- Paz, P.-G., & M., F. J. (2024). A review and classification on distributed permutation flowshop scheduling problems. *European Journal of Operational Research*, 312(1), 1–21.
- Perea, F., Yepes-Borrero, J. C., & Menezes, M. B. C. (2023). Acceptance ordering scheduling problem: The impact of an order-portfolio on a make-to-order firm's profitability. *International Journal of Production Economics*, 264.
- Rahmaniani, R., Crainic, T. G., Gendreau, M., & Rei, W. (2016). The benders decomposition algorithm: A literature review. *European Journal of Operational Research*, 259(3), 801–817.
- Riahi, V., Newton, M. A. H., Polash, M. M. A., & Sattar, A. (2019). Tailoring customer order scheduling search algorithms. *Computers & Operations Research*, 108, 155–165.
- Rom, W. O., & Slotnick, S. A. (2008). Order acceptance using genetic algorithms. *Computers & Operations Research*, 36(6), 1758–1767.
- Saab, A. (2021). Strengthening of feasibility cuts in logic-based benders decomposition. In *Proceedings of constraint programming, artificial intelligence, and* (p. 63). Springer.
- Sampathi, D. K. S., & Damodaran, P. (2024). Order acceptance and detailed scheduling in a job shop with discrete and batch processing machines using heuristics. In *IIEE annual conference. proceedings* (pp. 1–6). Institute of Industrial and Systems Engineers (IIEE).
- Shabtay, D., Gaspar, N., & Kaspi, M. (2015). A survey on offline scheduling with rejection (vol 16, pg 3, 2013). *Journal of Scheduling*, 18(3), 329–329.
- Slotnick, S. A. (2011). Order acceptance and scheduling: A taxonomy and review. *European Journal of Operational Research*, 212(1), 1–11.
- Wang, X., Xie, X., & Cheng, T. C. E. (2013). Order acceptance and scheduling in a two-machine flowshop. *International Journal of Production Economics*, 141(1), 366–376.
- Xiong, F., Chen, S., Xiong, N., & Jing, L. (2025). Scheduling distributed heterogeneous non-permutation flowshop to minimize the total weighted tardiness. *Expert Systems with Applications*, Article 126713.
- Yeung, W., Oğuz, C., & Cheng, T. E. (2004). Two-stage flowshop earliness and tardiness machine scheduling involving a common due window. *International Journal of Production Economics*, 90(3), 421–434.
- Zhang, C., Han, Y., Wang, Y., Li, J., & Gao, K. (2025). An accelerated discrete artificial bee colony algorithm under the makespan constraint: Solving the distributed blocking flow shop scheduling problem with balanced energy consumption costs. *Expert Systems with Applications*, 260, Article 125374.
- Zhang, Z.-Q., Hu, R., Qian, B., Jin, H.-P., Wang, L., & Yang, J.-B. (2022). A matrix cube-based estimation of distribution algorithm for the energy-efficient distributed assembly permutation flow-shop scheduling problem. *Expert Systems with Applications*, 194.
- Zhang, Z.-Q., Zhu, X.-P., Xu, Y.-X., Qian, B., Yang, B., & Hu, R. (2025). MEDHEA: Multidimensional estimation of distribution based hyper-heuristic evolutionary algorithm for energy-efficient distributed assembly no-wait flow-shop scheduling problem. *Expert Systems with Applications*, 271, Article 126526.
- Zhao, F., Yin, F., Wang, L., & Yu, Y. (2024). A co-evolution algorithm with dueling reinforcement learning mechanism for the energy-aware distributed heterogeneous flexible flow-shop scheduling problem. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*.
- Zhou, Z., Che, A., & Yan, P. (2012). A mixed integer programming approach for multi-cyclic robotic flowshop scheduling with time window constraints. *Applied Mathematical Modelling*, 36(8), 3621–3629.
- Zuo, Y., Zhao, F., & Yu, Y. (2025). Two-stage learning scatter search algorithm for the distributed hybrid flow shop scheduling problem with machine breakdown. *Expert Systems with Applications*, 259, Article 125344.